

C H A P T E R

17

SPECIALIZED LIBRARY FACILITIES

CONTENTS

Section 17.1 The <code>tuple</code> Type	718
Section 17.2 The <code>bitset</code> Type	723
Section 17.3 Regular Expressions	728
Section 17.4 Random Numbers	745
Section 17.5 The IO Library Revisited	752
Chapter Summary	769
Defined Terms	769

The latest standard greatly increased the size and scope of the library. Indeed, the portion of the standard devoted to the library more than doubled between the first release in 1998 and the 2011 standard. As a result, covering every C++ library class is well beyond the scope of this Primer. However, there are four library facilities that, although more specialized than other library facilities we’ve covered, are general enough to warrant discussion in an introductory book: `tuples`, `bitsets`, random-number generation, and regular expressions. In addition, we will also cover some additional, special-purpose parts of the IO library.

The library constitutes nearly two-thirds of the text of the new standard. Although we cannot cover every library facility in depth, there remain a few library facilities that are likely to be of use in many applications: tuples, bitsets, regular expressions, and random numbers. We'll also look at some additional IO library capabilities: format control, unformatted IO, and random access.

17.1 The tuple Type

A **tuple** is a template that is similar to a pair (§ 11.2.3, p. 426). Each pair type has different types for its members, but every pair always has exactly two members. A tuple also has members whose types vary from one tuple type to another, but a tuple can have any number of members. Each distinct tuple type has a fixed number of members, but the number of members in one tuple type can differ from the number of members in another.

A tuple is most useful when we want to combine some data into a single object but do not want to bother to define a data structure to represent those data. Table 17.1 lists the operations that tuples support. The tuple type, along with its companion types and functions, are defined in the tuple header.



A tuple can be thought of as a “quick and dirty” data structure.

17.1.1 Defining and Initializing tuples

When we define a tuple, we name the type(s) of each of its members:

```
tuple<size_t, size_t, size_t> threeD; // all three members set to 0
tuple<string, vector<double>, int, list<int>>
    someVal("constants", {3.14, 2.718}, 42, {0,1,2,3,4,5});
```

When we create a tuple object, we can use the default tuple constructor, which value initializes (§ 3.3.1, p. 98) each member, or we can supply an initializer for each member as we do in the initialization of `someVal`. This tuple constructor is explicit (§ 7.5.4, p. 296), so we must use the direct initialization syntax:

```
tuple<size_t, size_t, size_t> threeD = {1,2,3}; // error
tuple<size_t, size_t, size_t> threeD{1,2,3};    // ok
```

Alternatively, similar to the `make_pair` function (§ 11.2.3, p. 428), the library defines a `make_tuple` function that generates a tuple object:

```
// tuple that represents a bookstore transaction: ISBN, count, price per book
auto item = make_tuple("0-999-78345-X", 3, 20.00);
```

Like `make_pair`, the `make_tuple` function uses the types of the supplied initializers to infer the type of the tuple. In this case, `item` is a tuple whose type is `tuple<const char*,int,double>`.

Table 17.1: Operations on tuples

<code>tuple<T1, T2, ..., Tn> t;</code>	<code>t</code> is a tuple with as many members as there are types <code>T1 ... Tn</code> . The members are value initialized (§ 3.3.1, p. 98).
<code>tuple<T1, T2, ..., Tn> t(v1, v2, ..., vn);</code>	<code>t</code> is a tuple with types <code>T1 ... Tn</code> in which each member is initialized from the corresponding initializer, <code>v_i</code> . This constructor is explicit (§ 7.5.4, p. 296).
<code>make_tuple(v1, v2, ..., vn)</code>	Returns a tuple initialized from the given initializers. The type of the tuple is inferred from the types of the initializers.
<code>t1 == t2</code>	Two tuples are equal if they have the same number of members and if each pair of members are equal. Uses each member's underlying <code>==</code> operator. Once a member is found to be unequal, subsequent members are not tested.
<code>t1 != t2</code>	
<code>t1 relop t2</code>	Relational operations on tuples using dictionary ordering (§ 9.2.7, p. 340). The tuples must have the same number of members. Members of <code>t1</code> are compared with the corresponding members from <code>t2</code> using the <code><</code> operator
<code>get<i>(t)</code>	Returns a reference to the <i>i</i> th data member of <code>t</code> ; if <code>t</code> is an lvalue, the result is an lvalue reference; otherwise, it is an rvalue reference. All members of a tuple are public.
<code>tuple_size<tupleType>::value</code>	A class template that can be instantiated by a tuple type and has a public constexpr static data member named <code>value</code> of type <code>size_t</code> that is number of members in the specified tuple type.
<code>tuple_element<i, tupleType>::type</code>	A class template that can be instantiated by an integral constant and a tuple type and has a public member named <code>type</code> that is the type of the specified members in the specified tuple type.

Accessing the Members of a tuple

A pair always has two members, which makes it possible for the library to give these members names (i.e., `first` and `second`). No such naming convention is possible for `tuple` because there is no limit on the number of members a tuple type can have. As a result, the members are unnamed. Instead, we access the members of a tuple through a library function template named `get`. To use `get` we must specify an explicit template argument (§ 16.2.2, p. 682), which is the position of the member we want to access. We pass a tuple object to `get`, which returns a reference to the specified member:

```
auto book = get<0>(item);           // returns the first member of item
auto cnt = get<1>(item);           // returns the second member of item
auto price = get<2>(item)/cnt;     // returns the last member of item
get<2>(item) *= 0.8;               // apply 20% discount
```

The value inside the brackets must be an integral constant expression (§ 2.4.4, p. 65). As usual, we count from 0, meaning that `get<0>` is the first member.

If we have a tuple whose precise type details we don't know, we can use two auxiliary class templates to find the number and types of the tuple's members:

```

typedef decltype(item) trans; // trans is the type of item
// returns the number of members in object's of type trans
size_t sz = tuple_size<trans>::value; // returns 3
// cnt has the same type as the second member in item
tuple_element<1, trans>::type cnt = get<1>(item); // cnt is an int

```

To use `tuple_size` or `tuple_element`, we need to know the type of a tuple object. As usual, the easiest way to determine an object's type is to use `decltype` (§ 2.5.3, p. 70). Here, we use `decltype` to define a type alias for the type of `item`, which we use to instantiate both templates.

`tuple_size` has a public static data member named `value` that is the number of members in the specified tuple. The `tuple_element` template takes an index as well as a tuple type. `tuple_element` has a public type member named `type` that is the type of the specified member of the specified tuple type. Like `get`, `tuple_element` uses indices starting at 0.

Relational and Equality Operators

The tuple relational and equality operators behave similarly to the corresponding operations on containers (§ 9.2.7, p. 340). These operators execute pairwise on the members of the left-hand and right-hand tuples. We can compare two tuples only if they have the same number of members. Moreover, to use the equality or inequality operators, it must be legal to compare each pair of members using the `==` operator; to use the relational operators, it must be legal to use `<`. For example:

```

tuple<string, string> duo("1", "2");
tuple<size_t, size_t> twoD(1, 2);
bool b = (duo == twoD); // error: can't compare a size_t and a string

tuple<size_t, size_t, size_t> threeD(1, 2, 3);
b = (twoD < threeD);    // error: differing number of members

tuple<size_t, size_t> origin(0, 0);
b = (origin < twoD);    // ok: b is true

```



Because tuple defines the `<` and `==` operators, we can pass sequences of tuples to the algorithms and can use a tuple as key type in an ordered container.

EXERCISES SECTION 17.1.1

Exercise 17.1: Define a tuple that holds three `int` values and initialize the members to 10, 20, and 30.

Exercise 17.2: Define a tuple that holds a `string`, a `vector<string>`, and a `pair<string, int>`.

Exercise 17.3: Rewrite the `TextQuery` programs from § 12.3 (p. 484) to use a tuple instead of the `QueryResult` class. Explain which design you think is better and why.

17.1.2 Using a tuple to Return Multiple Values

A common use of tuple is to return multiple values from a function. For example, our bookstore might be one of several stores in a chain. Each store would have a transaction file that holds data on each book that the store recently sold. We might want to look at the sales for a given book in all the stores.

We'll assume that we have a file of transactions for each store. Each of these per-store transaction files will contain all the transactions for each book grouped together. We'll further assume that some other function reads these transaction files, builds a vector<Sales_data> for each store, and puts those vectors in a vector of vectors:

```
// each element in files holds the transactions for a particular store
vector<vector<Sales_data>> files;
```

We'll write a function that will search files looking for the stores that sold a given book. For each store that has a matching transaction, we'll create a tuple to hold the index of that store and two iterators. The index will be the position of the matching store in files. The iterators will mark the first and one past the last record for the given book in that store's vector<Sales_data>.

A Function That Returns a tuple

We'll start by writing the function to find a given book. This function's arguments are the vector of vectors just described, and a string that represents the book's ISBN. Our function will return a vector of tuples that will have an entry for each store with at least one sale for the given book:

```
// matches has three members: an index of a store and iterators into that store's vector
typedef tuple<vector<Sales_data>::size_type,
            vector<Sales_data>::const_iterator,
            vector<Sales_data>::const_iterator> matches;

// files holds the transactions for every store
// findBook returns a vector with an entry for each store that sold the given book
vector<matches>
findBook(const vector<vector<Sales_data>> &files,
        const string &book)
{
    vector<matches> ret; // initially empty
    // for each store find the range of matching books, if any
    for (auto it = files.cbegin(); it != files.cend(); ++it) {
        // find the range of Sales_data that have the same ISBN
        auto found = equal_range(it->cbegin(), it->cend(),
                                book, compareIsbn);
        if (found.first != found.second) // this store had sales
            // remember the index of this store and the matching range
            ret.push_back(make_tuple(it - files.cbegin(),
                                    found.first, found.second));
    }
    return ret; // empty if no matches found
}
```

The `for` loop iterates through the elements in `files`. Those elements are themselves vectors. Inside the `for` we call a library algorithm named `equal_range`, which operates like the associative container member of the same name (§ 11.3.5, p. 439). The first two arguments to `equal_range` are iterators denoting an input sequence (§ 10.1, p. 376). The third argument is a value. By default, `equal_range` uses the `<` operator to compare elements. Because `Sales_data` does not have a `<` operator, we pass a pointer to the `compareIsbn` function (§ 11.2.2, p. 425).

The `equal_range` algorithm returns a pair of iterators that denote a range of elements. If `book` is not found, then the iterators will be equal, indicating that the range is empty. Otherwise, the first member of the returned pair will denote the first matching transaction and `second` will be one past the last.

Using a tuple Returned by a Function

Once we have built our vector of stores with matching transactions, we need to process these transactions. In this program, we'll report the total sales results for each store that has a matching sale:

```
void reportResults(istream &in, ostream &os,
                  const vector<vector<Sales_data>> &files)
{
    string s;    // book to look for
    while (in >> s) {
        auto trans = findBook(files, s); // stores that sold this book
        if (trans.empty()) {
            cout << s << " not found in any stores" << endl;
            continue; // get the next book to look for
        }
        for (const auto &store : trans) // for every store with a sale
            // get<n> returns the specified member from the tuple in store
            os << "store " << get<0>(store) << " sales: "
               << accumulate(get<1>(store), get<2>(store),
                              Sales_data(s))
               << endl;
    }
}
```

The `while` loop repeatedly reads the `istream` named `in` to get the next book to process. We call `findBook` to see if `s` is present, and assign the results to `trans`. We use `auto` to simplify writing the type of `trans`, which is a vector of tuples.

If `trans` is empty, there were no sales for `s`. In this case, we print a message and return to the `while` to get the next book to look for.

The `for` loop binds `store` to each element in `trans`. Because we don't intend to change the elements in `trans`, we declare `store` as a reference to `const`. We use `get` to print the relevant data: `get<0>` is the index of the corresponding store, `get<1>` is the iterator denoting the first transaction, and `get<2>` is the iterator one past the last.

Because `Sales_data` defines the addition operator (§ 14.3, p. 560), we can use the library `accumulate` algorithm (§ 10.2.1, p. 379) to sum the transactions. We

pass a `Sales_data` object initialized by the `Sales_data` constructor that takes a string (§ 7.1.4, p. 264) as the starting point for the summation. That constructor initializes the `bookNo` member from the given string and the `units_sold` and `revenue` members to zero.

EXERCISES SECTION 17.1.2

Exercise 17.4: Write and test your own version of the `findBook` function.

Exercise 17.5: Rewrite `findBook` to return a pair that holds an index and a pair of iterators.

Exercise 17.6: Rewrite `findBook` so that it does not use `tuple` or `pair`.

Exercise 17.7: Explain which version of `findBook` you prefer and why.

Exercise 17.8: What would happen if we passed `Sales_data()` as the third parameter to `accumulate` in the last code example in this section?

17.2 The `bitset` Type

In § 4.8 (p. 152) we covered the built-in operators that treat an integral operand as a collection of bits. The standard library defines the `bitset` class to make it easier to use bit operations and possible to deal with collections of bits that are larger than the longest integral type. The `bitset` class is defined in the `bitset` header.

17.2.1 Defining and Initializing `bitsets`

Table 17.2 (overleaf) lists the constructors for `bitset`. The `bitset` class is a class template that, like the `array` class, has a fixed size (§ 9.2.4, p. 336). When we define a `bitset`, we say how many bits the `bitset` will contain:

```
bitset<32> bitvec(1U); // 32 bits; low-order bit is 1, remaining bits are 0
```

The size must be a constant expression (§ 2.4.4, p. 65). This statement defines `bitvec` as a `bitset` that holds 32 bits. Just as with the elements of a vector, the bits in a `bitset` are not named. Instead, we refer to them positionally. The bits are numbered starting at 0. Thus, `bitvec` has bits numbered 0 through 31. The bits starting at 0 are referred to as the **low-order** bits, and those ending at 31 are referred to as **high-order** bits.

Initializing a `bitset` from an unsigned Value

When we use an integral value as an initializer for a `bitset`, that value is converted to unsigned long long and is treated as a bit pattern. The bits in the `bitset` are a copy of that pattern. If the size of the `bitset` is greater than the number of bits in an unsigned long long, then the remaining high-order bits

Table 17.2: Ways to Initialize a `bitset`

<code>bitset<n> b;</code>	<code>b</code> has <code>n</code> bits; each bit is 0. This constructor is a <code>constexpr</code> (§ 7.5.6, p. 299).
<code>bitset<n> b(u);</code>	<code>b</code> is a copy of the <code>n</code> low-order bits of unsigned long long value <code>u</code> . If <code>n</code> is greater than the size of an unsigned long long, the high-order bits beyond those in the unsigned long long are set to zero. This constructor is a <code>constexpr</code> (§ 7.5.6, p. 299).
<code>bitset<n> b(s, pos, m, zero, one);</code>	<code>b</code> is a copy of the <code>m</code> characters from the string <code>s</code> starting at position <code>pos</code> . <code>s</code> may contain only the characters <code>zero</code> and <code>one</code> ; if <code>s</code> contains any other character, throws <code>invalid_argument</code> . The characters are stored in <code>b</code> as <code>zero</code> and <code>one</code> , respectively. <code>pos</code> defaults to 0, <code>m</code> defaults to <code>string::npos</code> , <code>zero</code> defaults to '0', and <code>one</code> defaults to '1'.
<code>bitset<n> b(cp, pos, m, zero, one);</code>	Same as the previous constructor, but copies from the character array to which <code>cp</code> points. If <code>m</code> is not supplied, then <code>cp</code> must point to a C-style string. If <code>m</code> is supplied, there must be at least <code>m</code> characters that are <code>zero</code> or <code>one</code> starting at <code>cp</code> .
The constructors that take a <code>string</code> or character pointer are <code>explicit</code> (§ 7.5.4, p. 296). The ability to specify alternate characters for 0 and 1 was added in the new standard.	

are set to zero. If the size of the `bitset` is less than that number of bits, then only the low-order bits from the given value are used; the high-order bits beyond the size of the `bitset` object are discarded:

```
// bitvec1 is smaller than the initializer; high-order bits from the initializer are discarded
bitset<13> bitvec1(0xbeef); // bits are 1111011101111
// bitvec2 is larger than the initializer; high-order bits in bitvec2 are set to zero
bitset<20> bitvec2(0xbeef); // bits are 00001011111011101111
// on machines with 64-bit long long 0ULL is 64 bits of 0, so ~0ULL is 64 ones
bitset<128> bitvec3(~0ULL); // bits 0...63 are one; 63...127 are zero
```

Initializing a `bitset` from a `string`

We can initialize a `bitset` from either a `string` or a pointer to an element in a character array. In either case, the characters represent the bit pattern directly. As usual, when we use strings to represent numbers, the characters with the lowest indices in the string correspond to the high-order bits, and vice versa:

```
bitset<32> bitvec4("1100"); // bits 2 and 3 are 1, all others are 0
```

If the string contains fewer characters than the size of the `bitset`, the high-order bits are set to zero.

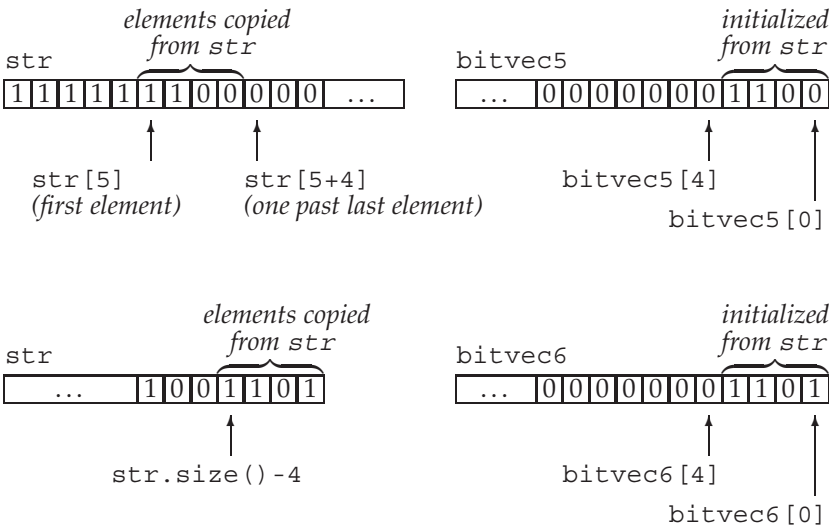


The indexing conventions of strings and `bitsets` are inversely related: The character in the `string` with the highest subscript (the right-most character) is used to initialize the low-order bit in the `bitset` (the bit with subscript 0). When you initialize a `bitset` from a `string`, it is essential to remember this difference.

We need not use the entire string as the initial value for the `bitset`. Instead, we can use a substring as the initializer:

```
string str("1111111000000011001101");  
bitset<32> bitvec5(str, 5, 4); // four bits starting at str[5], 1100  
bitset<32> bitvec6(str, str.size()-4); // use last four characters
```

Here `bitvec5` is initialized by the substring in `str` starting at `str[5]` and continuing for four positions. As usual, the right-most character of the substring represents the lowest-order bit. Thus, `bitvec5` is initialized with bit positions 3 through 0 set to 1100 and the remaining bits set to 0. The initializer for `bitvec6` passes a string and a starting point, so `bitvec6` is initialized from the characters in `str` starting four from the end of `str`. The remainder of the bits in `bitvec6` are initialized to zero. We can view these initializations as



EXERCISES SECTION 17.2.1

Exercise 17.9: Explain the bit pattern each of the following `bitset` objects contains:

- (a) `bitset<64> bitvec(32);`
- (b) `bitset<32> bv(1010101);`
- (c) `string bstr; cin >> bstr; bitset<8>bv(bstr);`

17.2.2 Operations on `bitsets`

The `bitset` operations (Table 17.3 (overleaf)) define various ways to test or set one or more bits. The `bitset` class also supports the bitwise operators that we covered in § 4.8 (p. 152). The operators have the same meaning when applied to `bitset` objects as the built-in operators have when applied to unsigned operands.

Table 17.3: `bitset` Operations

<code>b.any()</code>	Is any bit in <code>b</code> on?
<code>b.all()</code>	Are all the bits in <code>b</code> on?
<code>b.none()</code>	Are no bits in <code>b</code> on?
<code>b.count()</code>	Number of bits in <code>b</code> that are on.
<code>b.size()</code>	A <code>constexpr</code> function (§ 2.4.4, p. 65) that returns the number of bits in <code>b</code> .
<code>b.test(pos)</code>	Returns <code>true</code> if bit at position <code>pos</code> is on, <code>false</code> otherwise.
<code>b.set(pos, v)</code>	Sets the bit at position <code>pos</code> to the <code>bool</code> value <code>v</code> . <code>v</code> defaults to <code>true</code> . If no arguments, turns on all the bits in <code>b</code> .
<code>b.set()</code>	
<code>b.reset(pos)</code>	Turns off the bit at position <code>pos</code> or turns off all the bits in <code>b</code> .
<code>b.reset()</code>	
<code>b.flip(pos)</code>	Changes the state of the bit at position <code>pos</code> or of every bit in <code>b</code> .
<code>b.flip()</code>	
<code>b[pos]</code>	Gives access to the bit in <code>b</code> at position <code>pos</code> ; if <code>b</code> is <code>const</code> , then <code>b[pos]</code> returns a <code>bool</code> value <code>true</code> if the bit is on, <code>false</code> otherwise.
<code>b.to_ulong()</code>	Returns an unsigned long or an unsigned long long with the same bits as in <code>b</code> . Throws <code>overflow_error</code> if the bit pattern in <code>b</code> won't fit in the indicated result type.
<code>b.to_ullong()</code>	
<code>b.to_string(zero, one)</code>	Returns a string representing the bit pattern in <code>b</code> . <code>zero</code> and <code>one</code> default to <code>'0'</code> and <code>'1'</code> and are used to represent the bits 0 and 1 in <code>b</code> .
<code>os << b</code>	Prints the bits in <code>b</code> as the characters 1 or 0 to the stream <code>os</code> .
<code>is >> b</code>	Reads characters from <code>is</code> into <code>b</code> . Reading stops when the next character is not a 1 or 0 or when <code>b.size()</code> bits have been read.

Several operations—`count`, `size`, `all`, `any`, and `none`—take no arguments and return information about the state of the entire `bitset`. Others—`set`, `reset`, and `flip`—change the state of the `bitset`. The members that change the `bitset` are overloaded. In each case, the version that takes no arguments applies the given operation to the entire set; the versions that take a position apply the operation to the given bit:

```

bitset<32> bitvec(1U); // 32 bits; low-order bit is 1, remaining bits are 0
bool is_set = bitvec.any(); // true, one bit is set
bool is_not_set = bitvec.none(); // false, one bit is set
bool all_set = bitvec.all(); // false, only one bit is set
size_t onBits = bitvec.count(); // returns 1
size_t sz = bitvec.size(); // returns 32

bitvec.flip(); // reverses the value of all the bits in bitvec
bitvec.reset(); // sets all the bits to 0
bitvec.set(); // sets all the bits to 1

```

The `any` operation returns `true` if one or more bits of the `bitset` object are turned on—that is, are equal to 1. Conversely, `none` returns `true` if all the bits are zero. The new standard introduced the `all` operation, which returns `true` if all the bits are on. The `count` and `size` operations return a `size_t` (§ 3.5.2, p. 116) equal to

the number of bits that are set, or the total number of bits in the object, respectively. The `size` function is a `constexpr` and so can be used where a constant expression is required (§ 2.4.4, p. 65).

The `flip`, `set`, `reset`, and `test` members let us read or write the bit at a given position:

```
bitvec.flip(0);    // reverses the value of the first bit
bitvec.set(bitvec.size() - 1); // turns on the last bit
bitvec.set(0, 0); // turns off the first bit
bitvec.reset(i);  // turns off the ith bit
bitvec.test(0);   // returns false because the first bit is off
```

The subscript operator is overloaded on `const`. The `const` version returns a `bool` value `true` if the bit at the given index is on, `false` otherwise. The `nonconst` version returns a special type defined by `bitset` that lets us manipulate the bit value at the given index position:

```
bitvec[0] = 0;           // turn off the bit at position 0
bitvec[31] = bitvec[0]; // set the last bit to the same value as the first bit
bitvec[0].flip();        // flip the value of the bit at position 0
~bitvec[0];              // equivalent operation; flips the bit at position 0
bool b = bitvec[0];      // convert the value of bitvec[0] to bool
```

Retrieving the Value of a `bitset`

The `to_ulong` and `to_ullong` operations return a value that holds the same bit pattern as the `bitset` object. We can use these operations only if the size of the `bitset` is less than or equal to the corresponding size, unsigned long for `to_ulong` and unsigned long long for `to_ullong`:

```
unsigned long ulong = bitvec3.to_ulong();
cout << "ulong = " << ulong << endl;
```



These operations throw an `overflow_error` exception (§ 5.6, p. 193) if the value in the `bitset` does not fit in the specified type.

`bitset` IO Operators

The input operator reads characters from the input stream into a temporary object of type `string`. It reads until it has read as many characters as the size of the corresponding `bitset`, or it encounters a character other than 1 or 0, or it encounters end-of-file or an input error. The `bitset` is then initialized from that temporary `string` (§ 17.2.1, p. 724). If fewer characters are read than the size of the `bitset`, the high-order bits are, as usual, set to 0.

The output operator prints the bit pattern in a `bitset` object:

```
bitset<16> bits;
cin >> bits;    // read up to 16 1 or 0 characters from cin
cout << "bits: " << bits << endl; // print what we just read
```

Using `bitsets`

To illustrate using `bitsets`, we'll reimplement the grading code from § 4.8 (p. 154) that used an unsigned long to represent the pass/fail quiz results for 30 students:

```
bool status;
// version using bitwise operators
unsigned long quizA = 0;           // this value is used as a collection of bits
quizA |= 1UL << 27;               // indicate student number 27 passed
status = quizA & (1UL << 27);    // check how student number 27 did
quizA &= ~(1UL << 27);           // student number 27 failed

// equivalent actions using the bitset library
bitset<30> quizB;                 // allocate one bit per student; all bits initialized to 0
quizB.set(27);                   // indicate student number 27 passed
status = quizB[27];              // check how student number 27 did
quizB.reset(27);                 // student number 27 failed
```

EXERCISES SECTION 17.2.2

Exercise 17.10: Using the sequence 1, 2, 3, 5, 8, 13, 21, initialize a `bitset` that has a 1 bit in each position corresponding to a number in this sequence. Default initialize another `bitset` and write a small program to turn on each of the appropriate bits.

Exercise 17.11: Define a data structure that contains an integral object to track responses to a true/false quiz containing 10 questions. What changes, if any, would you need to make in your data structure if the quiz had 100 questions?

Exercise 17.12: Using the data structure from the previous question, write a function that takes a question number and a value to indicate a true/false answer and updates the quiz results accordingly.

Exercise 17.13: Write an integral object that contains the correct answers for the true/false quiz. Use it to generate grades on the quiz for the data structure from the previous two exercises.

17.3 Regular Expressions

A **regular expression** is a way of describing a sequence of characters. Regular expressions are a stunningly powerful computational device. However, describing the languages used to define regular expressions is well beyond the scope of this Primer. Instead, we'll focus on how to use the C++ regular-expression library (RE library), which is part of the new library. The RE library, which is defined in the `regex` header, involves several components, listed in Table 17.4.

C++
11



If you are not already familiar with using regular expressions, you might want to skim this section to get an idea of the kinds of things regular expressions can do.

Table 17.4: Regular Expression Library Components

<code>regex</code>	Class that represents a regular expression
<code>regex_match</code>	Matches a sequence of characters against a regular expression
<code>regex_search</code>	Finds the first subsequence that matches the regular expression
<code>regex_replace</code>	Replaces a regular expression using a given format
<code>sregex_iterator</code>	Iterator adaptor that calls <code>regex_search</code> to iterate through the matches in a string
<code>smatch</code>	Container class that holds the results of searching a string
<code>ssub_match</code>	Results for a matched subexpression in a string

The **regex** class represents a regular expression. Aside from initialization and assignment, `regex` has few operations. The operations on `regex` are listed in Table 17.6 (p. 731).

The functions **regex_match** and **regex_search** determine whether a given character sequence matches a given `regex`. The `regex_match` function returns `true` if the entire input sequence matches the expression; `regex_search` returns `true` if there is a substring in the input sequence that matches. There is also a `regex_replace` function that we'll describe in § 17.3.4 (p. 741).

The arguments to the `regex` functions are described in Table 17.5 (overleaf). These functions return a `bool` and are overloaded: One version takes an additional argument of type **smatch**. If present, these functions store additional information about a successful match in the given `smatch` object.

17.3.1 Using the Regular Expression Library

As a fairly simple example, we'll look for words that violate a well-known spelling rule of thumb, "*i* before *e* except after *c*":

```
// find the characters ei that follow a character other than c
string pattern("[^c]ei");
// we want the whole word in which our pattern appears
pattern = "[[:alpha:]]*" + pattern + "[[:alpha:]]*";
regex r(pattern); // construct a regex to find pattern
smatch results;   // define an object to hold the results of a search
// define a string that has text that does and doesn't match pattern
string test_str = "receipt freind theif receive";
// use r to find a match to pattern in test_str
if (regex_search(test_str, results, r)) // if there is a match
    cout << results.str() << endl;    // print the matching word
```

We start by defining a string to hold the regular expression we want to find. The regular expression `[^c]` says we want any character that is not a '`c`', and `[^c]ei` says we want any such letter that is followed by the letters `ei`. This pattern describes strings containing exactly three characters. We want the entire word that contains this pattern. To match the word, we need a regular expression that will match the letters that come before and after our three-letter pattern.

Table 17.5: Arguments to `regex_search` and `regex_match`

Note: These operations return `bool` indicating whether a match was found.

`(seq, m, r, mft)` Look for the regular expression in the `regex` object `r` in the character sequence `seq`. `seq` can be a `string`, a pair of iterators denoting a range, or a pointer to a null-terminated character array.
`(seq, r, mft)` `m` is a `match` object, which is used to hold details about the match.
`m` and `seq` must have compatible types (see § 17.3.1 (p. 733)).
`mft` is an optional `regex_constants::match_flag_type` value.
These values, listed in Table 17.13 (p. 744), affect the match process.

That regular expression consists of zero or more letters followed by our original three-letter pattern followed by zero or more additional characters. By default, the regular-expression language used by `regex` objects is ECMAScript. In ECMAScript, the pattern `[[:alpha:]]` matches any alphabetic character, and the symbols `+` and `*` signify that we want “one or more” or “zero or more” matches, respectively. Thus, `[[:alpha:]]*` will match zero or more characters.

Having stored our regular expression in `pattern`, we use it to initialize a `regex` object named `r`. We next define a `string` that we’ll use to test our regular expression. We initialize `test_str` with words that match our pattern (e.g., “freind” and “theif”) and words (e.g., “receipt” and “receive”) that don’t. We also define an `smatch` object named `results`, which we will pass to `regex_search`. If a match is found, `results` will hold the details about where the match occurred.

Next we call `regex_search`. If `regex_search` finds a match, it returns `true`. We use the `str` member of `results` to print the part of `test_str` that matched our pattern. The `regex_search` function stops looking as soon as it finds a matching substring in the input sequence. Thus, the output will be

`freind`

§ 17.3.2 (p. 734) will show how to find all the matches in the input.

Specifying Options for a `regex` Object

When we define a `regex` or call `assign` on a `regex` to give it a new value, we can specify one or more flags that affect how the `regex` operates. These flags control the processing done by that object. The last six flags listed in Table 17.6 indicate the language in which the regular expression is written. Exactly one of the flags that specify a language must be set. By default, the `ECMAScript` flag is set, which causes the `regex` to use the ECMA-262 specification, which is the regular expression language that many Web browsers use.

The other three flags let us specify language-independent aspects of the regular-expression processing. For example, we can indicate that we want the regular expression to be matched in a case-independent manner.

As one example, we can use the `icase` flag to find file names that have a particular file extension. Most operating systems recognize extensions in a case-independent manner—we can store a C++ program in a file that ends in `.cc`, or

Table 17.6: `regex` (and `wregex`) Operations

<code>regex r(re)</code>	<i>re</i> represents a regular expression and can be a <code>string</code> , a pair of iterators denoting a range of characters, a pointer to a null-terminated character array, a character pointer and a count, or a braced list of characters. <i>f</i> are flags that specify how the object will execute. <i>f</i> is set from the values listed below. If <i>f</i> is not specified, it defaults to <code>ECMAScript</code> .
<code>regex r(re, f)</code>	
<code>r1 = re</code>	Replace the regular expression in <i>r1</i> with <i>re</i> . <i>re</i> represents a regular expression and can be another <code>regex</code> , a <code>string</code> , a pointer to a null-terminated character array, or a braced list of characters.
<code>r1.assign(re, f)</code>	Same effect as using the assignment operator (<code>=</code>). <i>re</i> and optional flag <i>f</i> same as corresponding arguments to <code>regex</code> constructors.
<code>r.mark_count()</code>	Number of subexpressions (which we'll cover in § 17.3.3 (p. 738)) in <i>r</i> .
<code>r.flags()</code>	Returns the flags set for <i>r</i> .
<i>Note: Constructors and assignment operations may throw exceptions of type <code>regex_error</code>.</i>	
Flags Specified When a <code>regex</code> Is Defined	
Defined in <code>regex</code> and <code>regex_constants::syntax_option_type</code>	
<code>icase</code>	Ignore case during the match
<code>nosubs</code>	Don't store subexpression matches
<code>optimize</code>	Favor speed of execution over speed of construction
<code>ECMAScript</code>	Use grammar as specified by ECMA-262
<code>basic</code>	Use POSIX basic regular-expression grammar
<code>extended</code>	Use POSIX extended regular-expression grammar
<code>awk</code>	Use grammar from the POSIX version of the <i>awk</i> language
<code>grep</code>	Use grammar from the POSIX version of <i>grep</i>
<code>egrep</code>	Use grammar from the POSIX version of <i>egrep</i>

`.Cc`, or `.cC`, or `.CC`. We'll write a regular expression to recognize any of these along with other common file extensions as follows:

```
// one or more alphanumeric characters followed by a '.' followed by "cpp" or "cxx" or "cc"
regex r("[[:alnum:]]+\\. (cpp|cxx|cc)$", regex::icase);
smatch results;
string filename;
while (cin >> filename)
    if (regex_search(filename, results, r))
        cout << results.str() << endl; // print the current match
```

This expression will match a string of one or more letters or digits followed by a period and followed by one of three file extensions. The regular expression will match the file extensions regardless of case.

Just as there are special characters in C++ (§ 2.1.3, p. 39), regular-expression languages typically also have special characters. For example, the dot (`.`) character usually matches any character. As we do in C++, we can escape the special nature of a character by preceding it with a backslash. Because the backslash is also a special character in C++, we must use a second backslash inside a string literal to indicate to C++ that we want a backslash. Hence, we must write `\\.` to represent a regular expression that will match a period.

Errors in Specifying or Using a Regular Expression

We can think of a regular expression as itself a “program” in a simple programming language. That language is not interpreted by the C++ compiler. Instead, a regular expression is “compiled” at run time when a `regex` object is initialized with or assigned a new pattern. As with any programming language, it is possible that the regular expressions we write can have errors.



It is important to realize that the syntactic correctness of a regular expression is evaluated at run time.

If we make a mistake in writing a regular expression, then at *run time* the library will throw an exception (§ 5.6, p. 193) of type `regex_error`. Like the standard exception types, `regex_error` has a `what` operation that describes the error that occurred (§ 5.6.2, p. 195). A `regex_error` also has a member named `code` that returns a numeric code corresponding to the type of error that was encountered. The values `code` returns are implementation defined. The standard errors that the RE library can throw are listed in Table 17.7.

For example, we might inadvertently omit a bracket in a pattern:

```
try {
    // error: missing close bracket after a1num; the constructor will throw
    regex r("[[:alnum:]]+\\.(cpp|cxx|cc)$", regex::icase);
} catch (regex_error e)
{ cout << e.what() << "\ncode: " << e.code() << endl; }
```

When run on our system, this program generates

```
regex_error(error_brack):
The expression contained mismatched [ and ].
code: 4
```

Table 17.7: Regular Expression Error Conditions

Defined in <code>regex</code> and in <code>regex_constants::error_type</code>	
<code>error_collate</code>	Invalid collating element request
<code>error_ctype</code>	Invalid character class
<code>error_escape</code>	Invalid escape character or trailing escape
<code>error_backref</code>	Invalid back reference
<code>error_brack</code>	Mismatched bracket ([or])
<code>error_paren</code>	Mismatched parentheses ((or))
<code>error_brace</code>	Mismatched brace ({ or })
<code>error_badbrace</code>	Invalid range inside a { }
<code>error_range</code>	Invalid character range (e.g., [z-a])
<code>error_space</code>	Insufficient memory to handle this regular expression
<code>error_badrepeat</code>	A repetition character (*, ?, +, or {) was not preceded by a valid regular expression
<code>error_complexity</code>	The requested match is too complex
<code>error_stack</code>	Insufficient memory to evaluate a match

Our compiler defines the `code` member to return the position of the error as listed in Table 17.7, counting, as usual, from zero.

ADVICE: AVOID CREATING UNNECESSARY REGULAR EXPRESSIONS

As we’ve seen, the “program” that a regular expression represents is compiled at run time, not at compile time. Compiling a regular expression can be a surprisingly slow operation, especially if you’re using the extended regular-expression grammar or are using complicated expressions. As a result, constructing a `regex` object and assigning a new regular expression to an existing `regex` can be time-consuming. To minimize this overhead, you should try to avoid creating more `regex` objects than needed. In particular, if you use a regular expression in a loop, you should create it outside the loop rather than recompiling it on each iteration.

Regular Expression Classes and the Input Sequence Type

We can search any of several types of input sequence. The input can be ordinary `char` data or `wchar_t` data and those characters can be stored in a library string or in an array of `char` (or the wide character versions, `wstring` or array of `wchar_t`). The RE library defines separate types that correspond to these differing types of input sequences.

For example, the `regex` class holds regular expressions of type `char`. The library also defines a `wregex` class that holds type `wchar_t` and has all the same operations as `regex`. The only difference is that the initializers of a `wregex` must use `wchar_t` instead of `char`.

The `match` and iterator types (which we will cover in the following sections) are more specific. These types differ not only by the character type, but also by whether the sequence is in a library string or an array: `smatch` represents string input sequences; `cmatch`, character array sequences; `wsmatch`, wide string (`wstring`) input; and `wcmatch`, arrays of wide characters.

Table 17.8: Regular Expression Library Classes

If Input Sequence Has Type	Use Regular Expression Classes
string	<code>regex</code> , <code>smatch</code> , <code>ssub_match</code> , and <code>sregex_iterator</code>
const char*	<code>regex</code> , <code>cmatch</code> , <code>csub_match</code> , and <code>cregex_iterator</code>
wstring	<code>wregex</code> , <code>wsmatch</code> , <code>wssub_match</code> , and <code>wsregex_iterator</code>
const wchar_t*	<code>wregex</code> , <code>wcmatch</code> , <code>wcsub_match</code> , and <code>wcregex_iterator</code>

The important point is that the RE library types we use must match the type of the input sequence. Table 17.8 indicates which types correspond to which kinds of input sequences. For example:

```
regex r("[[:alnum:]]+\\.(cpp|cxx|cc)$", regex::icase);
smatch results; // will match a string input sequence, but not char*
if (regex_search("myfile.cc", results, r)) // error: char* input
    cout << results.str() << endl;
```

The (C++) compiler will reject this code because the type of the match argument and the type of the input sequence do not match. If we want to search a character array, then we must use a `cmatch` object:

```
cmatch results; // will match character array input sequences
if (regex_search("myfile.cc", results, r))
    cout << results.str() << endl; // print the current match
```

In general, our programs will use string input sequences and the corresponding string versions of the RE library components.

EXERCISES SECTION 17.3.1

Exercise 17.14: Write several regular expressions designed to trigger various errors. Run your program to see what output your compiler generates for each error.

Exercise 17.15: Write a program using the pattern that finds words that violate the “*i* before *e* except after *c*” rule. Have your program prompt the user to supply a word and indicate whether the word is okay or not. Test your program with words that do and do not violate the rule.

Exercise 17.16: What would happen if your `regex` object in the previous program were initialized with “`^[^c]ei`”? Test your program using that pattern to see whether your expectations were correct.

17.3.2 The Match and Regex Iterator Types

The program on page 729 that found violations of the “*i* before *e* except after *c*” grammar rule printed only the first match in its input sequence. We can get all the matches by using an `sregex_iterator`. The regex iterators are iterator adaptors (§ 9.6, p. 368) that are bound to an input sequence and a `regex` object. As described in Table 17.8 (on the previous page), there are specific regex iterator types that correspond to each of the different types of input sequences. The iterator operations are described in Table 17.9 (p. 736).

When we bind an `sregex_iterator` to a string and a `regex` object, the iterator is automatically positioned on the first match in the given string. That is, the `sregex_iterator` constructor calls `regex_search` on the given string and `regex`. When we dereference the iterator, we get an `smatch` object corresponding to the results from the most recent search. When we increment the iterator, it calls `regex_search` to find the next match in the input string.

Using an `sregex_iterator`

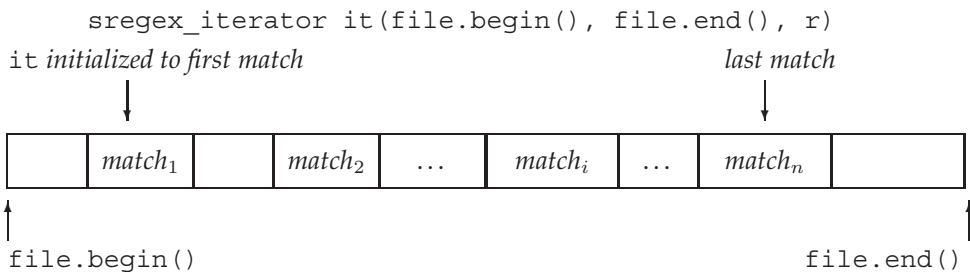
As an example, we’ll extend our program to find all the violations of the “*i* before *e* except after *c*” grammar rule in a file of text. We’ll assume that the string named `file` holds the entire contents of the input file that we want to search. This version of the program will use the same pattern as our original one, but will use a `sregex_iterator` to do the search:

```
// find the characters ei that follow a character other than c
string pattern("[^c]ei");
// we want the whole word in which our pattern appears
pattern = "[[:alpha:]]*" + pattern + "[[:alpha:]]*";
regex r(pattern, regex::icase); // we'll ignore case in doing the match
// it will repeatedly call regex_search to find all matches in file
for (sregex_iterator it(file.begin(), file.end(), r), end_it;
    it != end_it; ++it)
    cout << it->str() << endl; // matched word
```

The for loop iterates through each match to `r` inside `file`. The initializer in the for defines `it` and `end_it`. When we define `it`, the `sregex_iterator` constructor calls `regex_search` to position it on the first match in `file`. The empty `sregex_iterator`, `end_it`, acts as the off-the-end iterator. The increment in the for “advances” the iterator by calling `regex_search`. When we dereference the iterator, we get an `smatch` object representing the current match. We call the `str` member of the match to print the matching word.

We can think of this loop as jumping from match to match as illustrated in Figure 17.1.

Figure 17.1: Using an `sregex_iterator`



Using the Match Data

If we run this loop on `test_str` from our original program, the output would be

```
freind
theif
```

However, finding just the words that match our expression is not so useful. If we ran the program on a larger input sequence—for example, on the text of this chapter—we’d want to see the context within which the word occurs, such as

```
hey read or write according to the type
>>> being <<<
handled. The input operators ignore whi
```

In addition to letting us print the part of the input string that was matched, the match classes give us more detailed information about the match. The operations on these types are listed in Table 17.10 (p. 737) and Table 17.11 (p. 741).

Table 17.9: `sregex_iterator` Operations

These operations also apply to `cregex_iterator`,
`wsregex_iterator`, and `wcregex_iterator`

<code>sregex_iterator it(b, e, r);</code>	it is an <code>sregx_iterator</code> that iterates through the string denoted by iterators <code>b</code> and <code>e</code> . Calls <code>regex_search(b, e, r)</code> to position it on the first match in the input.
<code>sregex_iterator end;</code>	Off-the-end iterator for <code>sregex_iterator</code> .
<code>*it</code>	Returns a reference to the <code>smatch</code> object or a pointer to the <code>smatch</code> object
<code>it-></code>	from the most recent call to <code>regex_search</code> .
<code>++it</code>	Calls <code>regex_search</code> on the input sequence starting just after the current match. The prefix version returns a reference to the incremented iterator; postfix returns the old value.
<code>it1 == it2</code>	Two <code>sregex_iterator</code> s are equal if they are both the off-the-end iterator.
<code>it1 != it2</code>	Two non-end iterators are equal if they are constructed from the same input sequence and <code>regex</code> object.

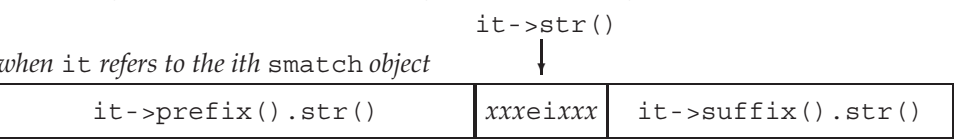
We'll have more to say about the `smatch` and `ssub_match` types in the next section. For now, what we need to know is that these types let us see the context of a match. The match types have members named `prefix` and `suffix`, which return a `ssub_match` object representing the part of the input sequence ahead of and after the current match, respectively. A `ssub_match` object has members named `str` and `length`, which return the matched string and size of that string, respectively. We can use these operations to rewrite the loop of our grammar program:

```
// same for loop header as before
for (sregex_iterator it(file.begin(), file.end(), r), end_it;
     it != end_it; ++it) {
    auto pos = it->prefix().length(); // size of the prefix
    pos = pos > 40 ? pos - 40 : 0;    // we want up to 40 characters
    cout << it->prefix().str().substr(pos) // last part of the prefix
         << "\n\t\t>>> " << it->str() << " <<<\n" // matched word
         << it->suffix().str().substr(0, 40) // first part of the suffix
         << endl;
}
```

The loop itself operates the same way as our previous program. What's changed is the processing inside the `for`, which is illustrated in Figure 17.2.

We call `prefix`, which returns an `ssub_match` object that represents the part of `file` ahead of the current match. We call `length` on that `ssub_match` to find out how many characters are in the part of `file` ahead of the match. Next we adjust `pos` to be the index of the character 40 from the end of the prefix. If the prefix has fewer than 40 characters, we set `pos` to 0, which means we'll print the entire prefix. We use `substr` (§ 9.5.1, p. 361) to print from the given position to the end of the prefix.

Figure 17.2: The `smatch` Object Representing a Particular Match



Having printed the characters that precede the match, we next print the match itself with some additional formatting so that the matched word will stand out in the output. After printing the matched portion, we print (up to) the first 40 characters in the part of `file` that comes after this match.

Table 17.10: smatch Operations	
These operations also apply to the <code>cmatch</code> , <code>wsmatch</code> , <code>wcmatch</code> and the corresponding <code>csub_match</code> , <code>wssub_match</code> , and <code>wcsub_match</code> types.	
<code>m.ready()</code>	true if <code>m</code> has been set by a call to <code>regex_search</code> or <code>regex_match</code> ; false otherwise. Operations on <code>m</code> are undefined if <code>ready</code> returns false.
<code>m.size()</code>	Zero if the match failed; otherwise, one plus the number of subexpressions in the most recently matched regular expression.
<code>m.empty()</code>	true if <code>m.size()</code> is zero.
<code>m.prefix()</code>	An <code>ssub_match</code> representing the sequence before the match.
<code>m.suffix()</code>	An <code>ssub_match</code> representing the part after the end of the match.
<code>m.format(...)</code>	See Table 17.12 (p. 742).
In the operations that take an index, <code>n</code> defaults to zero and must be less than <code>m.size()</code> . The first submatch (the one with index 0) represents the overall match.	
<code>m.length(n)</code>	Size of the <code>n</code> th matched subexpression.
<code>m.position(n)</code>	Distance of the <code>n</code> th subexpression from the start of the sequence.
<code>m.str(n)</code>	The matched string for the <code>n</code> th subexpression.
<code>m[n]</code>	<code>ssub_match</code> object corresponding to the <code>n</code> th subexpression.
<code>m.begin()</code> , <code>m.end()</code>	Iterators across the <code>sub_match</code> elements in <code>m</code> . As usual, <code>cbegin</code>
<code>m.cbegin()</code> , <code>m.cend()</code>	and <code>cend</code> return <code>const_iterator</code> s.

EXERCISES SECTION 17.3.2

Exercise 17.17: Update your program so that it finds all the words in an input sequence that violate the “ei” grammar rule.

Exercise 17.18: Revise your program to ignore words that contain “ei” but are not misspellings, such as “albeit” and “neighbor.”

17.3.3 Using Subexpressions

A pattern in a regular expression often contains one or more **subexpressions**. A subexpression is a part of the pattern that itself has meaning. Regular-expression grammars typically use parentheses to denote subexpressions.

As an example, the pattern that we used to match C++ files (§ 17.3.1, p. 730) used parentheses to group the possible file extensions. Whenever we group alternatives using parentheses, we are also declaring that those alternatives form a subexpression. We can rewrite that expression so that it gives us access to the file name, which is the part of the pattern that precedes the period, as follows:

```
// r has two subexpressions: the first is the part of the file name before the period
// the second is the file extension
regex r("([[:alnum:]]+)\. (cpp|cxx|cc)$", regex::icase);
```

Our pattern now has two parenthesized subexpressions:

- `([[:alnum:]]+)`, which is a sequence of one or more characters
- `(cpp| cxx| cc)`, which is the file extension

We can also rewrite the program from § 17.3.1 (p. 730) to print just the file name by changing the output statement:

```
if (regex_search(filename, results, r))
    cout << results.str(1) << endl; // print the first subexpression
```

As in our original program, we call `regex_search` to look for our pattern `r` in the string named `filename`, and we pass the `smatch` object `results` to hold the results of the match. If the call succeeds, then we print the results. However, in this program, we print `str(1)`, which is the match for the first subexpression.

In addition to providing information about the overall match, the match objects provide access to each matched subexpression in the pattern. The submatches are accessed positionally. The first submatch, which is at position 0, represents the match for the entire pattern. Each subexpression appears in order thereafter. Hence, the file name, which is the first subexpression in our pattern, is at position 1, and the file extension is in position 2.

For example, if the file name is `foo.cpp`, then `results.str(0)` will hold `foo.cpp`; `results.str(1)` will be `foo`; and `results.str(2)` will be `cpp`. In this program, we want the part of the name before the period, which is the first subexpression, so we print `results.str(1)`.

Subexpressions for Data Validation

One common use for subexpressions is to validate data that must match a specific format. For example, U.S. phone numbers have ten digits, consisting of an area code and a seven-digit local number. The area code is often, but not always, enclosed in parentheses. The remaining seven digits can be separated by a dash, a dot, or a space; or not separated at all. We might want to allow data with any of these formats and reject numbers in other forms. We'll do a two-step process: First,

we'll use a regular expression to find sequences that might be phone numbers and then we'll call a function to complete the validation of the data.

Before we write our phone number pattern, we need to describe a few more aspects of the ECMAScript regular-expression language:

- `\{d\}` represents a single digit and `\{d\}{n}` represents a sequence of *n* digits. (E.g., `\{d\}{3}` matches a sequence of three digits.)
- A collection of characters inside square brackets allows a match to any of those characters. (E.g., `[-. ]` matches a dash, a dot, or a space. Note that a dot has no special meaning inside brackets.)
- A component followed by `?` is optional. (E.g., `\{d\}{3}[-. ]?\{d\}{4}` matches three digits followed by an optional dash, period, or space, followed by four more digits. This pattern would match 555-0132 or 555.0132 or 555 0132 or 5550132.)
- Like C++, ECMAScript uses a backslash to indicate that a character should represent itself, rather than its special meaning. Because our pattern includes parentheses, which are special characters in ECMAScript, we must represent the parentheses that are part of our pattern as `\(` (or `\)`).

Because backslash is a special character in C++, each place that a `\` appears in the pattern, we must use a second backslash to indicate to C++ that we want a backslash. Hence, we write `\\{d\}{3}` to represent the regular expression `\{d\}{3}`.

In order to validate our phone numbers, we'll need access to the components of the pattern. For example, we'll want to verify that if a number uses an opening parenthesis for the area code, it also uses a close parenthesis after the area code. That is, we'd like to reject a number such as `(908.555.1800`.

To get at the components of the match, we need to define our regular expression using subexpressions. Each subexpression is marked by a pair of parentheses:

```
// our overall expression has seven subexpressions: ( ddd ) separator ddd separator dddd
// subexpressions 1, 3, 4, and 6 are optional; 2, 5, and 7 hold the number
"(\(\)?(\\d{3})\(\)\)?([- .])?(\\d{3})([- .]?)\\d{4}";
```

Because our pattern uses parentheses, and because we must escape backslashes, this pattern can be hard to read (and write!). The easiest way to read it is to pick off each (parenthesized) subexpression:

1. `(\(\)?` an optional open parenthesis for the area code
2. `(\\d{3})` the area code
3. `\(\)` an optional close parenthesis for the area code
4. `([- . ])?` an optional separator after the area code
5. `(\\d{3})` the next three digits of the number
6. `([- . ])?` another optional separator
7. `(\\d{4})` the final four digits of the number

The following code uses this pattern to read a file and find data that match our overall phone pattern. It will call a function named `valid` to check whether the number has a valid format:

```
string phone =
    "(\\(\\)?(\\d{3}) (\\d{3})?([- . ])?(\\d{3}) ([- . ])?(\\d{4})";
regex r(phone); // a regex to find our pattern
smatch m;
string s;
// read each record from the input file
while (getline(cin, s)) {
    // for each matching phone number
    for (sregex_iterator it(s.begin(), s.end(), r), end_it;
         it != end_it; ++it)
        // check whether the number's formatting is valid
        if (valid(*it))
            cout << "valid: " << it->str() << endl;
        else
            cout << "not valid: " << it->str() << endl;
}
```

Using the Submatch Operations

We'll use submatch operations, which are outlined in Table 17.11, to write the `valid` function. It is important to keep in mind that our pattern has seven subexpressions. As a result, each `smatch` object will contain eight `ssub_match` elements. The element at `[0]` represents the overall match; the elements `[1]... [7]` represent each of the corresponding subexpressions.

When we call `valid`, we know that we have an overall match, but we do not know which of our optional subexpressions were part of that match. The matched member of the `ssub_match` corresponding to a particular subexpression is `true` if that subexpression is part of the overall match.

In a valid phone number, the area code is either fully parenthesized or not parenthesized at all. Therefore, the work `valid` does depends on whether the number starts with a parenthesis or not:

```
bool valid(const smatch& m)
{
    // if there is an open parenthesis before the area code
    if (m[1].matched)
        // the area code must be followed by a close parenthesis
        // and followed immediately by the rest of the number or a space
        return m[3].matched
            && (m[4].matched == 0 || m[4].str() == " ");
    else
        // then there can't be a close after the area code
        // the delimiters between the other two components must match
        return !m[3].matched
            && m[4].str() == m[6].str();
}
```


Table 17.11: Submatch Operations

<i>Note: These operations apply to <code>ssub_match</code>, <code>csub_match</code>, <code>wssub_match</code>, <code>wcsub_match</code></i>	
<code>matched</code>	A public <code>bool</code> data member that indicates whether this <code>ssub_match</code> was matched.
<code>first</code>	public data members that are iterators to the start and one past the end of the
<code>second</code>	matching sequence. If there was no match, then <code>first</code> and <code>second</code> are equal.
<code>length()</code>	The size of this match. Returns 0 if <code>matched</code> is false.
<code>str()</code>	Returns a <code>string</code> containing the matched portion of the input. Returns the empty <code>string</code> if <code>matched</code> is false.
<code>s = ssub</code>	Convert the <code>ssub_match</code> object <code>ssub</code> to the <code>string</code> <code>s</code> . Equivalent to <code>s = ssub.str()</code> . The conversion operator is not explicit (§ 14.9.1, p. 581).

We start by checking whether the first subexpression (i.e., the open parenthesis) matched. That subexpression is in `m[1]`. If it matched, then the number starts with an open parenthesis. In this case, the overall number is valid if the subexpression following the area code also matched (meaning that there was a close parenthesis after the area code). Moreover, if the number is correctly parenthesized, then the next character must be a space or the first digit in the next part of the number.

If `m[1]` didn't match (i.e., there was no open parenthesis), the subexpression following the area code must also be empty. If it's empty, then the number is valid if the remaining separators are equal and not otherwise.

EXERCISES SECTION 17.3.3

- Exercise 17.19: Why is it okay to call `m[4].str()` without first checking whether `m[4]` was matched?
- Exercise 17.20: Write your own version of the program to validate phone numbers.
- Exercise 17.21: Rewrite your phone number program from § 8.3.2 (p. 323) to use the `valid` function defined in this section.
- Exercise 17.22: Rewrite your phone program so that it allows any number of white-space characters to separate the three parts of a phone number.
- Exercise 17.23: Write a regular expression to find zip codes. A zip code can have five or nine digits. The first five digits can be separated from the remaining four by a dash.

17.3.4 Using `regex_replace`

Regular expressions are often used when we need not only to find a given sequence but also to replace that sequence with another one. For example, we might want to translate U.S. phone numbers into the form “ddd.ddd.dddd,” where the area code and next three digits are separated by a dot.

When we want to find and replace a regular expression in the input sequence, we call `regex_replace`. Like the search functions, `regex_replace`, which is described in Table 17.12, takes an input character sequence and a `regex` object. We must also pass a string that describes the output we want.

We compose a replacement string by including the characters we want, intermixed with subexpressions from the matched substring. In this case, we want to use the second, fifth, and seventh subexpressions in our replacement string. We'll ignore the first, third, fourth, and sixth, because these were used in the original formatting of the number but are not part of our replacement format. We refer to a particular subexpression by using a `$` symbol followed by the index number for a subexpression:

```
string fmt = "$2.$5.$7"; // reformat numbers to ddd.ddd.dddd
```

We can use our regular-expression pattern and the replacement string as follows:

```
regex r(phone); // a regex to find our pattern
string number = "(908) 555-0132";
cout << regex_replace(number, r, fmt) << endl;
```

The output from this program is

```
908.555.0132
```

Table 17.12: Regular Expression Replace Operations

<code>m.format(dest, fmt, mft)</code> <code>m.format(fmt, mft)</code>	Produces formatted output using the format string <i>fmt</i> , the match in <i>m</i> , and the optional <code>match_flag_type</code> flags in <i>mft</i> . The first version writes to the output iterator <i>dest</i> (§ 10.5.1, p. 410) and takes <i>fmt</i> that is either a string or a pair of pointers denoting a range in a character array. The second version returns a string that holds the output and takes <i>fmt</i> that is a string or a pointer to a null-terminated character array. <i>mft</i> defaults to <code>format_default</code> .
<code>regex_replace</code> <code>(dest, seq, r, fmt, mft)</code> <code>regex_replace</code> <code>(seq, r, fmt, mft)</code>	Iterates through <i>seq</i> , using <code>regex_search</code> to find successive matches to <code>regex</code> <i>r</i> . Uses the format string <i>fmt</i> and optional <code>match_flag_type</code> flags in <i>mft</i> to produce its output. The first version writes to the output iterator <i>dest</i> , and takes a pair of iterators to denote <i>seq</i> . The second returns a string that holds the output and <i>seq</i> can be either a string or a pointer to a null-terminated character array. In all cases, <i>fmt</i> can be either a string or a pointer to a null-terminated character array, and <i>mft</i> defaults to <code>match_default</code> .

Replacing Only Part of the Input Sequence

A more interesting use of our regular-expression processing would be to replace phone numbers that are embedded in a larger file. For example, we might have a file of names and phone number that had data like this:

```
morgan (201) 555-0168 862-555-0123
drew (973) 555.0130
lee (609) 555-0132 2015550175 800.555-0100
```

that we want to transform to data like this:

```
morgan 201.555.0168 862.555.0123
drew 973.555.0130
lee 609.555.0132 201.555.0175 800.555.0100
```

We can generate this transformation with the following program:

```
int main()
{
    string phone =
        "\\(\\)?(\\d{3}) (\\d{3})?([- . ])?(\\d{3}) ([- . ])?(\\d{4}) ";
    regex r(phone); // a regex to find our pattern
    smatch m;
    string s;
    string fmt = "$2.$5.$7"; // reformat numbers to ddd.ddd.ddd
    // read each record from the input file
    while (getline(cin, s))
        cout << regex_replace(s, r, fmt) << endl;
    return 0;
}
```

We read each record into `s` and hand that record to `regex_replace`. This function finds and transforms *all* the matches in its input sequence.

Flags to Control Matches and Formatting

Just as the library defines flags to direct how to process a regular expression, the library also defines flags that we can use to control the match process or the formatting done during a replacement. These values are listed in Table 17.13 (overleaf). These flags can be passed to the `regex_search` or `regex_match` functions or to the format members of class `smatch`.

The match and format flags have type `match_flag_type`. These values are defined in a namespace named `regex_constants`. Like placeholders, which we used with `bind` (§ 10.3.4, p. 399), `regex_constants` is a namespace defined inside the `std` namespace. To use a name from `regex_constants`, we must qualify that name with the names of both namespaces:

```
using std::regex_constants::format_no_copy;
```

This declaration says that when our code uses `format_no_copy`, we want the object of that name from the namespace `std::regex_constants`. We can instead provide the alternative form of using that we will cover in § 18.2.2 (p. 792):

```
using namespace std::regex_constants;
```

Table 17.13: Match Flags

Defined in <code>regex_constants::match_flag_type</code>	
<code>match_default</code>	Equivalent to <code>format_default</code>
<code>match_not_bol</code>	Don't treat the first character as the beginning of the line
<code>match_not_eol</code>	Don't treat the last character as the end of the line
<code>match_not_bow</code>	Don't treat the first character as the beginning of a word
<code>match_not_eow</code>	Don't treat the last character as the end of a word
<code>match_any</code>	If there is more than one match, any match can be returned
<code>match_not_null</code>	Don't match an empty sequence
<code>match_continuous</code>	The match must begin with the first character in the input
<code>match_prev_avail</code>	The input sequence has characters before the first
<code>format_default</code>	Replacement string uses the ECMAScript rules
<code>format_sed</code>	Replacement string uses the rules from POSIX sed
<code>format_no_copy</code>	Don't output the unmatched parts of the input
<code>format_first_only</code>	Replace only the first occurrence

Using Format Flags

By default, `regex_replace` outputs its entire input sequence. The parts that don't match the regular expression are output without change; the parts that do match are formatted as indicated by the given format string. We can change this default behavior by specifying `format_no_copy` in the call to `regex_replace`:

```
// generate just the phone numbers: use a new format string
string fmt2 = "$2.$5.$7 "; // put space after the last number as a separator
// tell regex_replace to copy only the text that it replaces
cout << regex_replace(s, r, fmt2, format_no_copy) << endl;
```

Given the same input, this version of the program generates

```
201.555.0168 862.555.0123
973.555.0130
609.555.0132 201.555.0175 800.555.0100
```

EXERCISES SECTION 17.3.4

Exercise 17.24: Write your own version of the program to reformat phone numbers.

Exercise 17.25: Rewrite your phone program so that it writes only the first phone number for each person.

Exercise 17.26: Rewrite your phone program so that it writes only the second and subsequent phone numbers for people with more than one phone number.

Exercise 17.27: Write a program that reformats a nine-digit zip code as dddd-dddd.

17.4 Random Numbers

Programs often need a source of random numbers. Prior to the new standard, both C and C++ relied on a simple C library function named `rand`. That function produces pseudorandom integers that are uniformly distributed in the range from 0 to a system-dependent maximum value that is at least 32767.

C++
11

The `rand` function has several problems: Many, if not most, programs need random numbers in a different range from the one produced by `rand`. Some applications require random floating-point numbers. Some programs need numbers that reflect a nonuniform distribution. Programmers often introduce nonrandomness when they try to transform the range, type, or distribution of the numbers generated by `rand`.

The random-number library, defined in the `random` header, solves these problems through a set of cooperating classes: **random-number engines** and **random-number distribution classes**. These classes are described in Table 17.14. An engine generates a sequence of unsigned random numbers. A distribution uses an engine to generate random numbers of a specified type, in a given range, distributed according to a particular probability distribution.



C++ programs should not use the library `rand` function. Instead, they should use the `default_random_engine` along with an appropriate distribution object.

Table 17.14: Random Number Library Components

Engine	Types that generate a sequence of random unsigned integers
Distribution	Types that use an engine to return numbers according to a particular probability distribution

17.4.1 Random-Number Engines and Distribution

The random-number engines are function-object classes (§ 14.8, p. 571) that define a call operator that takes no arguments and returns a random unsigned number. We can generate raw random numbers by calling an object of a random-number engine type:

```
default_random_engine e; // generates random unsigned integers
for (size_t i = 0; i < 10; ++i)
    // e() "calls" the object to produce the next random number
    cout << e() << " ";
```

On our system, this program generates:

16807 282475249 1622650073 984943658 1144108930 470211272 ...

Here, we defined an object named `e` that has type `default_random_engine`. Inside the `for`, we call the object `e` to obtain the next random number.

The library defines several random-number engines that differ in terms of their performance and quality of randomness. Each compiler designates one of these engines as the `default_random_engine` type. This type is intended to be the engine with the most generally useful properties. Table 17.15 lists the engine operations and the engine types defined by the standard are listed in § A.3.2 (p. 884).

For most purposes, the output of an engine is not directly usable, which is why we described them earlier as raw random numbers. The problem is that the numbers usually span a range that differs from the one we need. *Correctly* transforming the range of a random number is surprisingly hard.

Distribution Types and Engines

To get a number in a specified range, we use an object of a distribution type:

```
// uniformly distributed from 0 to 9 inclusive
uniform_int_distribution<unsigned> u(0,9);
default_random_engine e; // generates unsigned random integers
for (size_t i = 0; i < 10; ++i)
    // u uses e as a source of numbers
    // each call returns a uniformly distributed value in the specified range
    cout << u(e) << " ";
```

This code produces output such as

```
0 1 7 4 5 2 0 6 6 9
```

Here we define `u` as a `uniform_int_distribution<unsigned>`. That type generates uniformly distributed unsigned values. When we define an object of this type, we can supply the minimum and maximum values we want. In this program, `u(0,9)` says that we want numbers to be in the range 0 to 9 *inclusive*. The random number distributions use inclusive ranges so that we can obtain every possible value of the given integral type.

Like the engine types, the distribution types are also function-object classes. The distribution types define a call operator that takes a random-number engine as its argument. The distribution object uses its engine argument to produce random numbers that the distribution object maps to the specified distribution.

Note that we pass the engine object itself, `u(e)`. Had we written the call as `u(e())`, we would have tried to pass the next value generated by `e` to `u`, which would be a compile-time error. We pass the engine, not the next result of the engine, because some distributions may need to call the engine more than once.



When we refer to a **random-number generator**, we mean the combination of a distribution object with an engine.

Comparing Random Engines and the `rand` Function

For readers familiar with the C library `rand` function, it is worth noting that the output of calling a `default_random_engine` object is similar to the output of `rand`. Engines deliver unsigned integers in a system-defined range. The range

for rand is 0 to RAND_MAX. The range for an engine type is returned by calling the min and max members on an object of that type:

```
cout << "min: " << e.min() << " max: " << e.max() << endl;
```

On our system this program produces the following output:

```
min: 1 max: 2147483646
```

Table 17.15: Random Number Engine Operations

Engine e;	Default constructor; uses the default seed for the engine type
Engine e(s);	Uses the integral value s as the seed
e.seed(s)	Reset the state of the engine using the seed s
e.min()	The smallest and largest numbers this generator will generate
e.max()	
Engine::result_type	The unsigned integral type this engine generates
e.discard(u)	Advance the engine by u steps; u has type unsigned long long

Engines Generate a Sequence of Numbers

Random number generators have one property that often confuses new users: Even though the numbers that are generated appear to be random, a given generator returns the same sequence of numbers each time it is run. The fact that the sequence is unchanging is very helpful during testing. On the other hand, programs that use random-number generators have to take this fact into account.

As one example, assume we need a function that will generate a vector of 100 random integers uniformly distributed in the range from 0 to 9. We might think we'd write this function as follows:

```
// almost surely the wrong way to generate a vector of random integers
// output from this function will be the same 100 numbers on every call!
vector<unsigned> bad_randVec()
{
    default_random_engine e;
    uniform_int_distribution<unsigned> u(0,9);
    vector<unsigned> ret;
    for (size_t i = 0; i < 100; ++i)
        ret.push_back(u(e));
    return ret;
}
```

However, this function will return the same vector every time it is called:

```
vector<unsigned> v1(bad_randVec());
vector<unsigned> v2(bad_randVec());
// will print equal
cout << ((v1 == v2) ? "equal" : "not equal") << endl;
```

This code will print `equal` because the vectors `v1` and `v2` have the same values.

The right way to write our function is to make the engine and associated distribution objects `static` (§ 6.1.1, p. 205):

```
// returns a vector of 100 uniformly distributed random numbers
vector<unsigned> good_randVec()
{
    // because engines and distributions retain state, they usually should be
    // defined as static so that new numbers are generated on each call
    static default_random_engine e;
    static uniform_int_distribution<unsigned> u(0,9);
    vector<unsigned> ret;
    for (size_t i = 0; i < 100; ++i)
        ret.push_back(u(e));
    return ret;
}
```

Because `e` and `u` are `static`, they will hold their state across calls to the function. The first call will use the first 100 random numbers from the sequence `u(e)` generates, the second call will get the next 100, and so on.



A given random-number generator always produces the same sequence of numbers. A function with a local random-number generator should make that generator (both the engine and distribution objects) `static`. Otherwise, the function will generate the identical sequence on each call.

Seeding a Generator

The fact that a generator returns the same sequence of numbers is helpful during debugging. However, once our program is tested, we often want to cause each run of the program to generate different random results. We do so by providing a **seed**. A seed is a value that an engine can use to cause it to start generating numbers at a new point in its sequence.

We can seed an engine in one of two ways: We can provide the seed when we create an engine object, or we can call the engine's `seed` member:

```
default_random_engine e1; // uses the default seed
default_random_engine e2(2147483646); // use the given seed value
// e3 and e4 will generate the same sequence because they use the same seed
default_random_engine e3; // uses the default seed value
e3.seed(32767); // call seed to set a new seed value
default_random_engine e4(32767); // set the seed value to 32767
for (size_t i = 0; i != 100; ++i) {
    if (e1() == e2())
        cout << "unseeded match at iteration: " << i << endl;
    if (e3() != e4())
        cout << "seeded differs at iteration: " << i << endl;
}
```

Here we define four engines. The first two, `e1` and `e2`, have different seeds and

should generate different sequences. The second two, `e3` and `e4`, have the same seed value. These two objects *will* generate the same sequence.

Picking a good seed, like most things about generating good random numbers, is surprisingly hard. Perhaps the most common approach is to call the system time function. This function, defined in the `ctime` header, returns the number of seconds since a given epoch. The `time` function takes a single parameter that is a pointer to a structure into which to write the time. If that pointer is null, the function just returns the time:

```
default_random_engine e1(time(0)); // a somewhat random seed
```

Because `time` returns time as the number of seconds, this seed is useful only for applications that generate the seed at second-level, or longer, intervals.



Using `time` as a seed usually doesn't work if the program is run repeatedly as part of an automated process; it might wind up with the same seed several times.

EXERCISES SECTION 17.4.1

Exercise 17.28: Write a function that generates and returns a uniformly distributed random unsigned int each time it is called.

Exercise 17.29: Allow the user to supply a seed as an optional argument to the function you wrote in the previous exercise.

Exercise 17.30: Revise your function again this time to take a minimum and maximum value for the numbers that the function should return.

17.4.2 Other Kinds of Distributions

The engines produce unsigned numbers, and each number in the engine's range has the same likelihood of being generated. Applications often need numbers of different types or distributions. The library handles both these needs by defining different distributions that, when used with an engine, produce the desired results. Table 17.16 (overleaf) lists the operations supported by the distribution types.

Generating Random Real Numbers

Programs often need a source of random floating-point values. In particular, programs frequently need random numbers between zero and one.

The most common, *but incorrect*, way to obtain a random floating-point from `rand` is to divide the result of `rand()` by `RAND_MAX`, which is a system-defined upper limit that is the largest random number that `rand` can return. This technique is incorrect because random integers usually have less precision than floating-point numbers, in which case there are some floating-point values that will never be produced as output.

With the new library facilities, we can easily obtain a floating-point random number. We define an object of type `uniform_real_distribution` and let the library handle mapping random integers to random floating-point numbers. As we did for `uniform_int_distribution`, we specify the minimum and maximum values when we define the object:

```
default_random_engine e; // generates unsigned random integers
// uniformly distributed from 0 to 1 inclusive
uniform_real_distribution<double> u(0,1);
for (size_t i = 0; i < 10; ++i)
    cout << u(e) << " ";
```

This code is nearly identical to the previous program that generated unsigned values. However, because we used a different distribution type, this version generates different results:

```
0.131538 0.45865 0.218959 0.678865 0.934693 0.519416 ...
```

Table 17.16: Distribution Operations

<i>Dist</i> d;	Default constructor; makes d ready to use. Other constructors depend on the type of <i>Dist</i> ; see § A.3 (p. 882). The distribution constructors are <code>explicit</code> (§ 7.5.4, p. 296).
d(e)	Successive calls with the same e produce a sequence of random numbers according to the distribution type of d; e is a random-number engine object.
d.min() d.max()	Return the smallest and largest numbers d(e) will generate.
d.reset()	Reestablish the state of d so that subsequent uses of d don't depend on values d has already generated.

Using the Distribution’s Default Result Type

With one exception, which we’ll cover in § 17.4.2 (p. 752), the distribution types are templates that have a single template type parameter that represents the type of the numbers that the distribution generates. These types always generate either a floating-point type or an integral type.

Each distribution template has a default template argument (§ 16.1.3, p. 670). The distribution types that generate floating-point values generate `double` by default. Distributions that generate integral results use `int` as their default. Because the distribution types have only one template parameter, when we want to use the default we must remember to follow the template’s name with empty angle brackets to signify that we want the default (§ 16.1.3, p. 671):

```
// empty <> signify we want to use the default result type
uniform_real_distribution<> u(0,1); // generates double by default
```

Generating Numbers That Are Not Uniformly Distributed

In addition to correctly generating numbers in a specified range, another advantage of the new library is that we can obtain numbers that are nonuniformly distributed. Indeed, the library defines 20 distribution types! These types are listed in § A.3 (p. 882).

As an example, we'll generate a series of normally distributed values and plot the resulting distribution. Because `normal_distribution` generates floating-point numbers, our program will use the `lround` function from the `cmath` header to round each result to its nearest integer. We'll generate 200 numbers centered around a mean of 4 with a standard deviation of 1.5. Because we're using a normal distribution, we can expect all but about 1 percent of the generated numbers to be in the range from 0 to 8, inclusive. Our program will count how many values appear that map to the integers in this range:

```
default_random_engine e;           // generates random integers
normal_distribution<> n(4,1.5);    // mean 4, standard deviation 1.5
vector<unsigned> vals(9);          // nine elements each 0
for (size_t i = 0; i != 200; ++i) {
    unsigned v = lround(n(e));      // round to the nearest integer
    if (v < vals.size())            // if this result is in range
        ++vals[v];                 // count how often each number appears
}
for (size_t j = 0; j != vals.size(); ++j)
    cout << j << ": " << string(vals[j], '*') << endl;
```

We start by defining our random generator objects and a `vector` named `vals`. We'll use `vals` to count how often each number in the range 0...9 occurs. Unlike most of our programs that use `vector`, we allocate `vals` at its desired size. By doing so, we start out with each element initialized to 0.

Inside the `for` loop, we call `lround(n(e))` to round the value returned by `n(e)` to the nearest integer. Having obtained the integer that corresponds to our floating-point random number, we use that number to index our vector of counters. Because `n(e)` can produce a number outside the range 0 to 9, we check that the number we got is in range before using it to index `vals`. If the number is in range, we increment the associated counter.

When the loop completes, we print the contents of `vals`, which will generate output such as

```
0: ***
1: *****
2: *****
3: *****
4: *****
5: *****
6: *****
7: *****
8: *
```

Here we print a `string` with as many asterisks as the count of the times the current value was returned by our random-number generator. Note that this figure

is not perfectly symmetrical. If it were, that symmetry should give us reason to suspect the quality of our random-number generator.

The `bernoulli_distribution` Class

We noted that there was one distribution that does not take a template parameter. That distribution is the `bernoulli_distribution`, which is an ordinary class, not a template. This distribution always returns a `bool` value. It returns `true` with a given probability. By default that probability is `.5`.

As an example of this kind of distribution, we might have a program that plays a game with a user. To play the game, one of the players—either the user or the program—has to go first. We could use a `uniform_int_distribution` object with a range of 0 to 1 to select the first player. Alternatively, we can use a `Bernoulli` distribution to make this choice. Assuming that we have a function named `play` that plays the game, we might have a loop such as the following to interact with the user:

```
string resp;
default_random_engine e; // e has state, so it must be outside the loop!
bernoulli_distribution b; // 50/50 odds by default
do {
    bool first = b(e); // if true, the program will go first
    cout << (first ? "We go first"
               : "You get to go first") << endl;
    // play the game passing the indicator of who goes first
    cout << ((play(first)) ? "sorry, you lost"
                  : "congrats, you won") << endl;
    cout << "play again? Enter 'yes' or 'no'" << endl;
} while (cin >> resp && resp[0] == 'y');
```

We use a `do while` (§ 5.4.4, p. 189) to repeatedly prompt the user to play.



WARNING

Because engines return the same sequence of numbers (§ 17.4.1, p. 747), it is essential that we declare engines outside of loops. Otherwise, we'd create a new engine on each iteration and generate the same values on each iteration. Similarly, distributions may retain state and should also be defined outside loops.

One reason to use a `bernoulli_distribution` in this program is that doing so lets us give the program a better chance of going first:

```
bernoulli_distribution b(.55); // give the house a slight edge
```

If we use this definition for `b`, then the program has 55/45 odds of going first.

17.5 The IO Library Revisited

In Chapter 8 we introduced the basic architecture and most commonly used parts of the IO library. In this section we'll look at three of the more specialized features that the IO library supports: format control, unformatted IO, and random access.

EXERCISES SECTION 17.4.2

Exercise 17.31: What would happen if we defined `b` and `e` inside the `do` loop of the game-playing program from this section?

Exercise 17.32: What would happen if we defined `resp` inside the loop?

Exercise 17.33: Write a version of the word transformation program from § 11.3.6 (p. 440) that allows multiple transformations for a given word and randomly selects which transformation to apply.

17.5.1 Formatted Input and Output

In addition to its condition state (§ 8.1.2, p. 312), each `iostream` object also maintains a format state that controls the details of how IO is formatted. The format state controls aspects of formatting such as the notational base for integral values, the precision of floating-point values, the width of an output element, and so on.

The library defines a set of **manipulators** (§ 1.2, p. 7), listed in Tables 17.17 (p. 757) and 17.18 (p. 760), that modify the format state of a stream. A manipulator is a function or object that affects the state of a stream and can be used as an operand to an input or output operator. Like the input and output operators, a manipulator returns the stream object to which it is applied, so we can combine manipulators and data in a single statement.

Our programs have already used one manipulator, `endl`, which we “write” to an output stream as if it were a value. But `endl` isn’t an ordinary value; instead, it performs an operation: It writes a newline and flushes the buffer.

Many Manipulators Change the Format State

Manipulators are used for two broad categories of output control: controlling the presentation of numeric values and controlling the amount and placement of padding. Most of the manipulators that change the format state provide set/unset pairs; one manipulator sets the format state to a new value and the other unsets it, restoring the normal default formatting.



Manipulators that change the format state of the stream usually leave the format state changed for all subsequent IO.

The fact that a manipulator makes a persistent change to the format state can be useful when we have a set of IO operations that want to use the same formatting. Indeed, some programs take advantage of this aspect of manipulators to reset the behavior of one or more formatting rules for all its input or output. In such cases, the fact that a manipulator changes the stream is a desirable property.

However, many programs (and, more importantly, programmers) expect the state of the stream to match the normal library defaults. In these cases, leaving the state of the stream in a nonstandard state can lead to errors. As a result, it is usually best to undo whatever state changes are made as soon as those changes are no longer needed.

Controlling the Format of Boolean Values

One example of a manipulator that changes the formatting state of its object is the `boolalpha` manipulator. By default, `bool` values print as 1 or 0. A `true` value is written as the integer 1 and a `false` value as 0. We can override this formatting by applying the `boolalpha` manipulator to the stream:

```
cout << "default bool values: " << true << " " << false
      << "\nalpha bool values: " << boolalpha
      << true << " " << false << endl;
```

When executed, this program generates the following:

```
default bool values: 1 0
alpha bool values: true false
```

Once we “write” `boolalpha` on `cout`, we’ve changed how `cout` will print `bool` values from this point on. Subsequent operations that print `bool`s will print them as either `true` or `false`.

To undo the format state change to `cout`, we apply `noboolalpha`:

```
bool bool_val = get_status();
cout << boolalpha      // sets the internal state of cout
      << bool_val
      << noboolalpha; // resets the internal state to default formatting
```

Here we change the format of `bool` values only to print the value of `bool_val`. Once that value is printed, we immediately reset the stream back to its initial state.

Specifying the Base for Integral Values

By default, integral values are written and read in decimal notation. We can change the notational base to octal or hexadecimal or back to decimal by using the manipulators `hex`, `oct`, and `dec`:

```
cout << "default: " << 20 << " " << 1024 << endl;
cout << "octal: " << oct << 20 << " " << 1024 << endl;
cout << "hex: " << hex << 20 << " " << 1024 << endl;
cout << "decimal: " << dec << 20 << " " << 1024 << endl;
```

When compiled and executed, this program generates the following output:

```
default: 20 1024
octal: 24 2000
hex: 14 400
decimal: 20 1024
```

Notice that like `boolalpha`, these manipulators change the format state. They affect the immediately following output and all subsequent integral output until the format is reset by invoking another manipulator.



The `hex`, `oct`, and `dec` manipulators affect only integral operands; the representation of floating-point values is unaffected.

Indicating Base on the Output

By default, when we print numbers, there is no visual cue as to what notational base was used. Is 20, for example, really 20, or an octal representation of 16? When we print numbers in decimal mode, the number is printed as we expect. If we need to print octal or hexadecimal values, it is likely that we should also use the `showbase` manipulator. The `showbase` manipulator causes the output stream to use the same conventions as used for specifying the base of an integral constant:

- A leading 0x indicates hexadecimal.
- A leading 0 indicates octal.
- The absence of either indicates decimal.

Here we've revised the previous program to use `showbase`:

```
cout << showbase;      // show the base when printing integral values
cout << "default: " << 20 << " " << 1024 << endl;
cout << "in octal: " << oct << 20 << " " << 1024 << endl;
cout << "in hex: " << hex << 20 << " " << 1024 << endl;
cout << "in decimal: " << dec << 20 << " " << 1024 << endl;
cout << noshowbase;    // reset the state of the stream
```

The revised output makes it clear what the underlying value really is:

```
default: 20 1024
in octal: 024 02000
in hex: 0x14 0x400
in decimal: 20 1024
```

The `noshowbase` manipulator resets `cout` so that it no longer displays the notational base of integral values.

By default, hexadecimal values are printed in lowercase with a lowercase x. We can display the X and the hex digits a–f as uppercase by applying the `uppercase` manipulator:

```
cout << uppercase << showbase << hex
    << "printed in hexadecimal: " << 20 << " " << 1024
    << nouppercase << noshowbase << dec << endl;
```

This statement generates the following output:

```
printed in hexadecimal: 0X14 0X400
```

We apply the `nouppercase`, `noshowbase`, and `dec` manipulators to return the stream to its original state.

Controlling the Format of Floating-Point Values

We can control three aspects of floating-point output:

- How many digits of precision are printed

- Whether the number is printed in hexadecimal, fixed decimal, or scientific notation
- Whether a decimal point is printed for floating-point values that are whole numbers

By default, floating-point values are printed using six digits of precision; the decimal point is omitted if the value has no fractional part; and they are printed in either fixed decimal or scientific notation depending on the value of the number. The library chooses a format that enhances readability of the number. Very large and very small values are printed using scientific notation. Other values are printed in fixed decimal.

Specifying How Much Precision to Print

By default, precision controls the total number of digits that are printed. When printed, floating-point values are rounded, not truncated, to the current precision. Thus, if the current precision is four, then 3.14159 becomes 3.142; if the precision is three, then it is printed as 3.14.

We can change the precision by calling the `precision` member of an IO object or by using the `setprecision` manipulator. The `precision` member is overloaded (§ 6.4, p. 230). One version takes an `int` value and sets the precision to that new value. It returns the *previous* precision value. The other version takes no arguments and returns the current precision value. The `setprecision` manipulator takes an argument, which it uses to set the precision.



The `setprecision` manipulators and other manipulators that take arguments are defined in the `iomanip` header.

The following program illustrates the different ways we can control the precision used to print floating-point values:

```
// cout.precision reports the current precision value
cout << "Precision: " << cout.precision()
    << ", Value: " << sqrt(2.0) << endl;

// cout.precision(12) asks that 12 digits of precision be printed
cout.precision(12);
cout << "Precision: " << cout.precision()
    << ", Value: " << sqrt(2.0) << endl;

// alternative way to set precision using the setprecision manipulator
cout << setprecision(3);
cout << "Precision: " << cout.precision()
    << ", Value: " << sqrt(2.0) << endl;
```

When compiled and executed, the program generates the following output:

```
Precision: 6, Value: 1.41421
Precision: 12, Value: 1.41421356237
Precision: 3, Value: 1.41
```


Table 17.17: Manipulators Defined in `iostream`

<code>boolalpha</code>	Display true and false as strings
* <code>noboolalpha</code>	Display true and false as 0, 1
<code>showbase</code>	Generate prefix indicating the numeric base of integral values
* <code>noshowbase</code>	Do not generate notational base prefix
<code>showpoint</code>	Always display a decimal point for floating-point values
* <code>noshowpoint</code>	Display a decimal point only if the value has a fractional part
<code>showpos</code>	Display + in nonnegative numbers
* <code>noshowpos</code>	Do not display + in nonnegative numbers
<code>uppercase</code>	Print 0X in hexadecimal, E in scientific
* <code>nouppercase</code>	Print 0x in hexadecimal, e in scientific
* <code>dec</code>	Display integral values in decimal numeric base
<code>hex</code>	Display integral values in hexadecimal numeric base
<code>oct</code>	Display integral values in octal numeric base
<code>left</code>	Add fill characters to the right of the value
<code>right</code>	Add fill characters to the left of the value
<code>internal</code>	Add fill characters between the sign and the value
<code>fixed</code>	Display floating-point values in decimal notation
<code>scientific</code>	Display floating-point values in scientific notation
<code>hexfloat</code>	Display floating-point values in hex (new to C++ 11)
<code>defaultfloat</code>	Reset the floating-point format to decimal (new to C++ 11)
<code>unitbuf</code>	Flush buffers after every output operation
* <code>nounitbuf</code>	Restore normal buffer flushing
* <code>skipws</code>	Skip whitespace with input operators
<code>noskipws</code>	Do not skip whitespace with input operators
<code>flush</code>	Flush the <code>ostream</code> buffer
<code>ends</code>	Insert null, then flush the <code>ostream</code> buffer
<code>endl</code>	Insert newline, then flush the <code>ostream</code> buffer

* indicates the default stream state

This program calls the library `sqrt` function, which is found in the `cmath` header. The `sqrt` function is overloaded and can be called on either a `float`, `double`, or `long double` argument. It returns the square root of its argument.

Specifying the Notation of Floating-Point Numbers



Unless you need to control the presentation of a floating-point number (e.g., to print data in columns or to print data that represents money or a percentage), it is usually best to let the library choose the notation.

We can force a stream to use scientific, fixed, or hexadecimal notation by using the appropriate manipulator. The `scientific` manipulator changes the stream to use scientific notation. The `fixed` manipulator changes the stream to use fixed decimal.

Under the new library, we can also force floating-point values to use hexadecimal format by using `hexfloat`. The new library provides another manipulator, named `defaultfloat`. This manipulator returns the stream to its default state in

C++
11

which it chooses a notation based on the value being printed.

These manipulators also change the default meaning of the precision for the stream. After executing `scientific`, `fixed`, or `hexfloat`, the precision value controls the number of digits after the decimal point. By default, precision specifies the total number of digits—both before and after the decimal point. Using `fixed` or `scientific` lets us print numbers lined up in columns, with the decimal point in a fixed position relative to the fractional part being printed:

```
cout << "default format: " << 100 * sqrt(2.0) << '\n'
    << "scientific: " << scientific << 100 * sqrt(2.0) << '\n'
    << "fixed decimal: " << fixed << 100 * sqrt(2.0) << '\n'
    << "hexadecimal: " << hexfloat << 100 * sqrt(2.0) << '\n'
    << "use defaults: " << defaultfloat << 100 * sqrt(2.0)
    << "\n\n";
```

produces the following output:

```
default format: 141.421
scientific: 1.414214e+002
fixed decimal: 141.421356
hexadecimal: 0x1.1ad7bcp+7
use defaults: 141.421
```

By default, the hexadecimal digits and the `e` used in scientific notation are printed in lowercase. We can use the uppercase manipulator to show those values in uppercase.

Printing the Decimal Point

By default, when the fractional part of a floating-point value is 0, the decimal point is not displayed. The `showpoint` manipulator forces the decimal point to be printed:

```
cout << 10.0 << endl;           // prints 10
cout << showpoint << 10.0       // prints 10.0000
    << noshowpoint << endl;     // revert to default format for the decimal point
```

The `noshowpoint` manipulator reinstates the default behavior. The next output expression will have the default behavior, which is to suppress the decimal point if the floating-point value has a 0 fractional part.

Padding the Output

When we print data in columns, we often need fairly fine control over how the data are formatted. The library provides several manipulators to help us accomplish the control we might need:

- `setw` to specify the minimum space for the *next* numeric or string value.
- `left` to left-justify the output.
- `right` to right-justify the output. Output is right-justified by default.

- internal controls placement of the sign on negative values. internal left-justifies the sign and right-justifies the value, padding any intervening space with blanks.
- `setfill` lets us specify an alternative character to use to pad the output. By default, the value is a space.



`setw`, like `endl`, does not change the internal state of the output stream. It determines the size of only the *next* output.

The following program illustrates these manipulators:

```
int i = -16;
double d = 3.14159;
// pad the first column to use a minimum of 12 positions in the output
cout << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n';
// pad the first column and left-justify all columns
cout << left
    << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n'
    << right; // restore normal justification
// pad the first column and right-justify all columns
cout << right
    << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n';
// pad the first column but put the padding internal to the field
cout << internal
    << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n';
// pad the first column, using # as the pad character
cout << setfill('#')
    << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n'
    << setfill(' '); // restore the normal pad character
```

When executed, this program generates

```
i:          -16next col
d:         3.14159next col
i: -16      next col
d: 3.14159  next col
i:          -16next col
d:         3.14159next col
i: -        16next col
d:         3.14159next col
i: -#####16next col
d: #####3.14159next col
```

Table 17.18: Manipulators Defined in <code>iomanip</code>	
<code>setfill(ch)</code>	Fill whitespace with <code>ch</code>
<code>setprecision(n)</code>	Set floating-point precision to <code>n</code>
<code>setw(w)</code>	Read or write value to <code>w</code> characters
<code>setbase(b)</code>	Output integers in base <code>b</code>

Controlling Input Formatting

By default, the input operators ignore whitespace (blank, tab, newline, formfeed, and carriage return). The following loop

```
char ch;
while (cin >> ch)
    cout << ch;
```

given the input sequence

```
a b    c
d
```

executes four times to read the characters `a` through `d`, skipping the intervening blanks, possible tabs, and newline characters. The output from this program is

```
abcd
```

The `noskipws` manipulator causes the input operator to read, rather than skip, whitespace. To return to the default behavior, we apply the `skipws` manipulator:

```
cin >> noskipws;    // set cin so that it reads whitespace
while (cin >> ch)
    cout << ch;
cin >> skipws;      // reset cin to the default state so that it discards whitespace
```

Given the same input as before, this loop makes seven iterations, reading white-space as well as the characters in the input. This loop generates

```
a b    c
d
```

EXERCISES SECTION 17.5.1

Exercise 17.34: Write a program that illustrates the use of each manipulator in Tables 17.17 (p. 757) and 17.18.

Exercise 17.35: Write a version of the program from page 758, that printed the square root of 2 but this time print hexadecimal digits in uppercase.

Exercise 17.36: Modify the program from the previous exercise to print the various floating-point values so that they line up in a column.

17.5.2 Unformatted Input/Output Operations

So far, our programs have used only **formatted IO** operations. The input and output operators (<< and >>) format the data they read or write according to the type being handled. The input operators ignore whitespace; the output operators apply padding, precision, and so on.

The library also provides a set of low-level operations that support **unformatted IO**. These operations let us deal with a stream as a sequence of uninterpreted bytes.

Single-Byte Operations

Several of the unformatted operations deal with a stream one byte at a time. These operations, which are described in Table 17.19, read rather than ignore whitespace. For example, we can use the unformatted IO operations `get` and `put` to read and write the characters one at a time:

```
char ch;
while (cin.get(ch))
    cout.put(ch);
```

This program preserves the whitespace in the input. Its output is identical to the input. It executes the same way as the previous program that used `noskipws`.

Table 17.19: Single-Byte Low-Level IO Operations

<code>is.get(ch)</code>	Put the next byte from the <code>istream is</code> in character <code>ch</code> . Returns <code>is</code> .
<code>os.put(ch)</code>	Put the character <code>ch</code> onto the <code>ostream os</code> . Returns <code>os</code> .
<code>is.get()</code>	Returns next byte from <code>is</code> as an <code>int</code> .
<code>is.putback(ch)</code>	Put the character <code>ch</code> back on <code>is</code> ; returns <code>is</code> .
<code>is.unget()</code>	Move <code>is</code> back one byte; returns <code>is</code> .
<code>is.peek()</code>	Return the next byte as an <code>int</code> but doesn't remove it.

Putting Back onto an Input Stream

Sometimes we need to read a character in order to know that we aren't ready for it. In such cases, we'd like to put the character back onto the stream. The library gives us three ways to do so, each of which has subtle differences from the others:

- `peek` returns a copy of the next character on the input stream but does not change the stream. The value returned by `peek` stays on the stream.
- `unget` backs up the input stream so that whatever value was last returned is still on the stream. We can call `unget` even if we do not know what value was last taken from the stream.
- `putback` is a more specialized version of `unget`: It returns the last value read from the stream but takes an argument that must be the same as the one that was last read.

In general, we are guaranteed to be able to put back at most one value before the next read. That is, we are not guaranteed to be able to call `putback` or `unget` successively without an intervening read operation.

int Return Values from Input Operations

The `peek` function and the version of `get` that takes no argument return a character from the input stream as an `int`. This fact can be surprising; it might seem more natural to have these functions return a `char`.

The reason that these functions return an `int` is to allow them to return an end-of-file marker. A given character set is allowed to use every value in the `char` range to represent an actual character. Thus, there is no extra value in that range to use to represent end-of-file.

The functions that return `int` convert the character they return to unsigned `char` and then promote that value to `int`. As a result, even if the character set has characters that map to negative values, the `int` returned from these operations will be a positive value (§ 2.1.2, p. 35). The library uses a negative value to represent end-of-file, which is thus guaranteed to be distinct from any legitimate character value. Rather than requiring us to know the actual value returned, the `cstdio` header defines a `const` named `EOF` that we can use to test if the value returned from `get` is end-of-file. It is essential that we use an `int` to hold the return from these functions:

```
int ch;      // use an int, not a char to hold the return from get ()
// loop to read and write all the data in the input
while ((ch = cin.get()) != EOF)
    cout.put(ch);
```

This program operates identically to the one on page 761, the only difference being the version of `get` that is used to read the input.

Multi-Byte Operations

Some unformatted IO operations deal with chunks of data at a time. These operations can be important if speed is an issue, but like other low-level operations, they are error-prone. In particular, these operations require us to allocate and manage the character arrays (§ 12.2, p. 476) used to store and retrieve data. The multi-byte operations are listed in Table 17.20.

The `get` and `getline` functions take the same parameters, and their actions are similar but not identical. In each case, `sink` is a `char` array into which the data are placed. The functions read until one of the following conditions occurs:

- `size - 1` characters are read
- End-of-file is encountered
- The delimiter character is encountered

The difference between these functions is the treatment of the delimiter: `get` leaves the delimiter as the next character of the `istream`, whereas `getline` reads and discards the delimiter. In either case, the delimiter is *not* stored in `sink`.

Table 17.20: Multi-Byte Low-Level IO Operations

<code>is.get(sink, size, delim)</code>	Reads up to <code>size</code> bytes from <code>is</code> and stores them in the character array beginning at the address pointed to by <code>sink</code> . Reads until encountering the <code>delim</code> character or until it has read <code>size</code> bytes or encounters end-of-file. If <code>delim</code> is present, it is left on the input stream and not read into <code>sink</code> .
<code>is.getline(sink, size, delim)</code>	Same behavior as the three-argument version of <code>get</code> but reads and discards <code>delim</code> .
<code>is.read(sink, size)</code>	Reads up to <code>size</code> bytes into the character array <code>sink</code> . Returns <code>is</code> .
<code>is.gcount()</code>	Returns number of bytes read from the stream <code>is</code> by the last call to an unformatted read operation.
<code>os.write(source, size)</code>	Writes <code>size</code> bytes from the character array <code>source</code> to <code>os</code> . Returns <code>os</code> .
<code>is.ignore(size, delim)</code>	Reads and ignores at most <code>size</code> characters up to and including <code>delim</code> . Unlike the other unformatted functions, <code>ignore</code> has default arguments: <code>size</code> defaults to 1 and <code>delim</code> to end-of-file.



It is a common error to intend to remove the delimiter from the stream but to forget to do so.

Determining How Many Characters Were Read

Several of the read operations read an unknown number of bytes from the input. We can call `gcount` to determine how many characters the last unformatted input operation read. It is essential to call `gcount` before any intervening unformatted input operation. In particular, the single-character operations that put characters back on the stream are also unformatted input operations. If `peek`, `unget`, or `putback` are called before calling `gcount`, then the return value will be 0.

17.5.3 Random Access to a Stream

The various stream types generally support random access to the data in their associated stream. We can reposition the stream so that it skips around, reading first the last line, then the first, and so on. The library provides a pair of functions to *seek* to a given location and to *tell* the current location in the associated stream.



Random IO is an inherently system-dependent. To understand how to use these features, you must consult your system's documentation.

Although these `seek` and `tell` functions are defined for all the stream types, whether they do anything useful depends on the device to which the stream is bound. On most systems, the streams bound to `cin`, `cout`, `cerr`, and `clog` do

CAUTION: LOW-LEVEL ROUTINES ARE ERROR-PRONE

In general, we advocate using the higher-level abstractions provided by the library. The IO operations that return `int` are a good example of why.

It is a common programming error to assign the return, from `get` or `peek` to a `char` rather than an `int`. Doing so is an error, but an error the compiler will not detect. Instead, what happens depends on the machine and on the input data. For example, on a machine in which `chars` are implemented as `unsigned chars`, this loop will run forever:

```
char ch;    // using a char here invites disaster!  
// the return from cin.get is converted to char and then compared to an int  
while ((ch = cin.get()) != EOF)  
    cout.put(ch);
```

The problem is that when `get` returns `EOF`, that value will be converted to an `unsigned char` value. That converted value is no longer equal to the `int` value of `EOF`, and the loop will continue forever. Such errors are likely to be caught in testing.

On machines for which `chars` are implemented as `signed chars`, we can't say with confidence what the behavior of the loop might be. What happens when an out-of-bounds value is assigned to a `signed` value is up to the compiler. On many machines, this loop will appear to work, unless a character in the input matches the `EOF` value. Although such characters are unlikely in ordinary data, presumably low-level IO is necessary only when we read binary values that do not map directly to ordinary characters and numeric values. For example, on our machine, if the input contains a character whose value is `'\377'`, then the loop terminates prematurely. `'\377'` is the value on our machine to which `-1` converts when used as a `signed char`. If the input has this value, then it will be treated as the (premature) end-of-file indicator.

Such bugs do not happen when we read and write typed values. If you can use the more type-safe, higher-level operations supported by the library, do so.

EXERCISES SECTION 17.5.2

Exercise 17.37: Use the unformatted version of `getline` to read a file a line at a time. Test your program by giving it a file that contains empty lines as well as lines that are longer than the character array that you pass to `getline`.

Exercise 17.38: Extend your program from the previous exercise to print each word you read onto its own line.

not support random access—after all, what would it mean to jump back ten places when we're writing directly to `cout`? We can call the `seek` and `tell` functions, but these functions will fail at run time, leaving the stream in an invalid state.



Because the `istream` and `ostream` types usually do not support random access, the remainder of this section should be considered as applicable to only the `fstream` and `sstream` types.

Seek and Tell Functions

To support random access, the IO types maintain a marker that determines where the next read or write will happen. They also provide two functions: One repositions the marker by *seeking* to a given position; the second *tells* us the current position of the marker. The library actually defines two pairs of *seek* and *tell* functions, which are described in Table 17.21. One pair is used by input streams, the other by output streams. The input and output versions are distinguished by a suffix that is either a *g* or a *p*. The *g* versions indicate that we are “getting” (reading) data, and the *p* functions indicate that we are “putting” (writing) data.

Table 17.21: Seek and Tell Functions

<code>tellg()</code>	Return the current position of the marker in an input stream (<code>tellg</code>)
<code>tellp()</code>	or an output stream (<code>tellp</code>).
<code>seekg(pos)</code>	Reposition the marker in an input or output stream to the given absolute address in the stream. <code>pos</code> is usually a value returned by a previous call to the corresponding <code>tellg</code> or <code>tellp</code> function.
<code>seekp(pos)</code>	
<code>seekp(off, from)</code>	Reposition the marker for an input or output stream integral number <code>off</code> characters ahead or behind <code>from</code> . <code>from</code> can be one of
<code>seekg(off, from)</code>	• <code>beg</code>, seek relative to the beginning of the stream • <code>cur</code>, seek relative to the current position of the stream • <code>end</code>, seek relative to the end of the stream

Logically enough, we can use only the *g* versions on an *istream* and on the types *ifstream* and *istringstream* that inherit from *istream* (§ 8.1, p. 311). We can use only the *p* versions on an *ostream* and on the types that inherit from it, *ofstream* and *ostringstream*. An *iostream*, *fstream*, or *stringstream* can both read and write the associated stream; we can use either the *g* or *p* versions on objects of these types.

There Is Only One Marker

The fact that the library distinguishes between the “putting” and “getting” versions of the seek and tell functions can be misleading. Even though the library makes this distinction, it maintains only a single marker in a stream—there is *not* a distinct read marker and write marker.

When we’re dealing with an input-only or output-only stream, the distinction isn’t even apparent. We can use only the *g* or only the *p* versions on such streams. If we attempt to call `tellp` on an *ifstream*, the compiler will complain. Similarly, it will not let us call `seekg` on an *ostringstream*.

The *fstream* and *stringstream* types can read and write the same stream. In these types there is a single buffer that holds data to be read and written and a single marker denoting the current position in the buffer. The library maps both the *g* and *p* positions to this single marker.



Because there is only a single marker, we *must* do a seek to reposition the marker whenever we switch between reading and writing.

Repositioning the Marker

There are two versions of the seek functions: One moves to an “absolute” address within the file; the other moves to a byte offset from a given position:

```
// set the marker to a fixed position
seekg(new_position);    // set the read marker to the given pos_type location
seekp(new_position);    // set the write marker to the given pos_type location

// offset some distance ahead of or behind the given starting point
seekg(offset, from);    // set the read marker offset distance from from
seekp(offset, from);    // offset has type off_type
```

The possible values for `from` are listed in Table 17.21 (on the previous page).

The arguments, `new_position` and `offset`, have machine-dependent types named `pos_type` and `off_type`, respectively. These types are defined in both `istream` and `ostream`. `pos_type` represents a file position and `off_type` represents an offset from that position. A value of type `off_type` can be positive or negative; we can seek forward or backward in the file.

Accessing the Marker

The `tellg` or `tellp` functions return a `pos_type` value denoting the current position of the stream. The tell functions are usually used to remember a location so that we can subsequently seek back to it:

```
// remember the current write position in mark
ostream& writeStr;    // output stringstream
ostream&::pos_type mark = writeStr.tellp();
// ...
if (cancelEntry)
    // return to the remembered position
    writeStr.seekp(mark);
```

Reading and Writing to the Same File

Let’s look at a programming example. Assume we are given a file to read. We are to write a newline at the end of the file that contains the relative position at which each line begins. For example, given the following file,

```
abcd
efg
hi
j
```

the program should produce the following modified file:

```

abcd
efg
hi
j
5 9 12 14

```

Note that our program need not write the offset for the first line—it always occurs at position 0. Also note that the offset counts must include the invisible newline character that ends each line. Finally, note that the last number in the output is the offset for the line on which our output begins. By including this offset in our output, we can distinguish our output from the file's original contents. We can read the last number in the resulting file and seek to the corresponding offset to get to the beginning of our output.

Our program will read the file a line at a time. For each line, we'll increment a counter, adding the size of the line we just read. That counter is the offset at which the next line starts:

```

int main()
{
    // open for input and output and preposition file pointers to end-of-file
    // file mode argument see § 8.4 (p. 319)
    fstream inOut("copyOut",
                  fstream::ate | fstream::in | fstream::out);
    if (!inOut) {
        cerr << "Unable to open file!" << endl;
        return EXIT_FAILURE; // EXIT_FAILURE see § 6.3.2 (p. 227)
    }

    // inOut is opened in ate mode, so it starts out positioned at the end
    auto end_mark = inOut.tellg(); // remember original end-of-file position
    inOut.seekg(0, fstream::beg); // reposition to the start of the file
    size_t cnt = 0; // accumulator for the byte count
    string line; // hold each line of input

    // while we haven't hit an error and are still reading the original data
    while (inOut && inOut.tellg() != end_mark
           && getline(inOut, line)) { // and can get another line of input
        cnt += line.size() + 1; // add 1 to account for the newline
        auto mark = inOut.tellg(); // remember the read position
        inOut.seekp(0, fstream::end); // set the write marker to the end
        inOut << cnt; // write the accumulated length
        // print a separator if this is not the last line
        if (mark != end_mark) inOut << " ";
        inOut.seekg(mark); // restore the read position
    }
    inOut.seekp(0, fstream::end); // seek to the end
    inOut << "\n"; // write a newline at end-of-file
    return 0;
}

```

Our program opens its `fstream` using the `in`, `out`, and `ate` modes (§ 8.4, p. 319). The first two modes indicate that we intend to read and write the same file. Speci-

fyng ate positions the read and write markers at the end of the file. As usual, we check that the open succeeded, and exit if it did not (§ 6.3.2, p. 227).

Because our program writes to its input file, we can't use end-of-file to signal when it's time to stop reading. Instead, our loop must end when it reaches the point at which the original input ended. As a result, we must first remember the original end-of-file position. Because we opened the file in `ate` mode, `inOut` is already positioned at the end. We store the current (i.e., the original end) position in `end_mark`. Having remembered the end position, we reposition the read marker at the beginning of the file by seeking to the position 0 bytes from the beginning of the file.

The `while` loop has a three-part condition: We first check that the stream is valid; if so, we check whether we've exhausted our original input by comparing the current read position (returned by `tellg`) with the position we remembered in `end_mark`. Finally, assuming that both tests succeeded, we call `getline` to read the next line of input. If `getline` succeeds, we perform the body of the loop.

The loop body starts by remembering the current position in `mark`. We save that position in order to return to it after writing the next relative offset. The call to `seekp` repositions the write marker to the end of the file. We write the counter value and then `seekg` back to the position we remembered in `mark`. Having re-stored the marker, we're ready to repeat the condition in the `while`.

Each iteration of the loop writes the offset of the next line. Therefore, the last iteration of the loop takes care of writing the offset of the last line. However, we still need to write a newline at the end of the file. As with the other writes, we call `seekp` to position the file at the end before writing the newline.

EXERCISES SECTION 17.5.3

Exercise 17.39: Write your own version of the `seek` program presented in this section.

CHAPTER SUMMARY

This chapter covered additional IO operations and four library types: `tuple`, `bitset`, regular expressions, and random numbers.

A `tuple` is a template that allows us to bundle together members of disparate types into a single object. Each `tuple` contains a specified number of members, but the library imposes no limit on the number of members we can define for a given `tuple` type.

A `bitset` lets us define collections of bits of a specified size. The size of a `bitset` is not constrained to match any of the integral types, and can even exceed them. In addition to supporting the normal bitwise operators (§ 4.8, p. 152), `bitset` defines a number of named operations that let us manipulate the state of particular bits in the `bitset`.

The regular-expression library provides a collection of classes and functions: The `regex` class manages regular expressions written in one of several common regular-expression languages. The match classes hold information about a specific match. These classes are used by the `regex_search` and `regex_match` functions. These functions take a `regex` object and a character sequence and detect whether the regular expression in that `regex` matches the given character sequence. The `regex` iterator types are iterator adaptors that use `regex_search` to iterate through an input sequence and return each matching subsequence. There is also a `regex_replace` function that lets us replace the matched part of a given input sequence with a specified alternative.

The random-number library is a collection of random-number engines and distribution classes. A random-number engine returns a sequence of uniformly distributed integral values. The library defines several engines that have different performance characteristics. The `default_random_engine` is defined as the engine that should be suitable for most casual uses. The library also defines 20 distribution types. These distribution types use an engine to deliver random numbers of a specified type in a given range that are distributed according to a specified probability distribution.

DEFINED TERMS

bitset Standard library class that holds a collection of bits of a size that is known at compile time, and provides operations to test and set the bits in the collection.

cmatch Container of `csub_match` objects that provides information about the match to a `regex` on `const char*` input sequences. The first element in the container describes the overall match results. The subsequent elements describe the results for the subexpressions.

cregex_iterator Like `sregex_iterator`

except that it iterates over an array of `char`.

csub_match Type that holds the results of a regular expression match to a `const char*`. Can represent the entire match or a subexpression.

default random engine Type alias for the random number engine intended for normal use.

formatted IO IO operations that use the types of the objects being read or written to define the actions of the operations. For-

matted input operations perform whatever transformations are appropriate to the type being read, such as converting ASCII numeric strings to the indicated arithmetic type and (by default) ignoring whitespace. Formatted output routines convert types to printable character representations, pad the output, and may perform other, type-specific transformations.

get Template function that returns the specified member for a given tuple. For example, `get<0>(t)` returns the first element from the tuple `t`.

high-order Bits in a `bitset` with the largest indices.

low-order Bits in a `bitset` with the lowest indices.

manipulator A function-like object that “manipulates” a stream. Manipulators can be used as the right-hand operand to the overloaded IO operators, `<<` and `>>`. Most manipulators change the internal state of the object. Such manipulators often come in pairs—one to change the state and the other to return the stream to its default state.

random-number distribution Standard library type that transforms the output of a random-number engine according to its named distribution. For example, `uniform_int_distribution<T>` generates uniformly distributed integers of type `T`, `normal_distribution<T>` generates normally distributed numbers, and so on.

random-number engine Library type that generates random unsigned numbers. Engines are intended to be used only as inputs to random-number distributions.

random-number generator Combination of a random-number engine type and a distribution type.

regex Class that manages a regular expression.

regex_error Exception type thrown to indicate a syntactic error in a regular expression.

regex_match Function that determines whether the entire input sequence matches the given `regex` object.

regex_replace Function that uses a `regex` object to replace matching subexpressions in an input sequence using a given format.

regex_search Function that uses a `regex` object to find a matching subsequence of a given input sequence.

regular expression A way of describing a sequence of characters.

seed Value supplied to a random-number engine that causes it to move to a new point in the sequence of number that it generates.

smatch Container of `ssub_match` objects that provides information about the match to a `regex` on string input sequences. The first element in the container describes the overall match results. The subsequent elements describe the results for the subexpressions.

sregex_iterator Iterator that iterates over a string using a given `regex` object to find matches in the given string. The constructor positions the iterator on the first match by calling `regex_search`. Incrementing the iterator calls `regex_search` starting just after the current match in the given string. Dereferencing the iterator returns an `smatch` object describing the current match.

ssub_match Type that holds results of a regular expression match to a string. Can represent the entire match or a subexpression.

subexpression Parenthesized component of a regular expression pattern.

tuple Template that generates types that hold unnamed members of specified types. There is no fixed limit on the number of members a `tuple` can be defined to have.

unformatted IO Operations that treat the stream as an undifferentiated byte stream. Unformatted operations place more of the burden for managing the IO on the user.